Exponential time hypothesis is false... but the verified algorithm is slower than trying all 2^n assignments at every practical size.

Alex Linhares
Argo Lab AGI; on leave from the Getulio Vargas Foundation
alex@argolab.org ORCID 0000-0002-4227-6879

November 5, 2026

Abstract

Placeholder. Item 09 writes the abstract (under 200 words; no number without a section that regenerates it). The title's sentence made precise: what is verified is a guarantee with existential constants and the exponent $n(\log\log n)^2/\sqrt{\log n}$, whose factor exceeds 1 for every $50 \le n < e^{5504}$, so the verified guarantee is weaker than the trivial $2^n \cdot \text{poly}$ guarantee at every practical size; the machine's running time is unknown because the constants are never computed; the manuscript's sharper exponents are unverified; the statement does refute ETH; the audit found no flaw.

1 Introduction

Placeholder. Item 09 drafts this section: the title's sentence made precise in the first paragraph (a guarantee, not a running time; existential constants never computed; $\rho(n) > n$ for every $50 \le n < e^{5504}$; the sharper exponents unverified); the one-paragraph summary; what is claimed and what is not; the stance (a careful reading by people who checked; no flaw found). Ends with what this does not claim".

2 The verified statement, read exactly

2.1 What we are reading, and the words we need

OpenAI's collection (OpenAI, 2026) holds, for the result it calls family 378, one machine-checked theorem. It is named `exact_count_and_sat`. It is stated in the family's challenge file, which is 220 lines long: `lean/ComparatorChallenges/SubexponentialSAT.lean` at commit `4eaee875` of the collection. This section reads that file and says what the theorem asserts and what it does not. Line numbers are those of the file. We quote nothing from memory.

A few words first, each in one sentence. *Lean* is a proof assistant: a program that checks every step of a proof and refuses a proof with a gap. The *kernel* is the small core of Lean that does the final checking; when we say a statement is verified, we mean the kernel accepted a proof of it. A *formula* here is a Boolean formula in conjunctive normal form: a list of *clauses*, each clause an "or"

of a few variables or negated variables, and the whole formula true when every clause is true. A *k-CNF* is such a formula whose clauses have at most k variables each; 3-SAT is the problem of deciding whether a 3-CNF can be made true. A *satisfying assignment* is a choice of true or false for every variable that makes the formula true; with n variables there are 2^n assignments to try, and *trying all* 2^n *assignments* is the method the title names. The *comparator* is OpenAI's checking tool. It takes a *challenge file*, which states a theorem and leaves the proof blank, and a *solution*, which proves it, and checks that the two state the same theorem and that the proof uses only three standard axioms. An axiom is a statement accepted without proof; the three standard ones are the ones every ordinary mathematical proof in Lean uses. The file we read is the challenge file. It carries the statement with the placeholder `sorry` where the proof would go (line 218). The solution proves the same statement in 3,091 files (Section 4). We rebuilt that proof ourselves and the kernel reported the three standard axioms and nothing else (Section 5).

2.2 The definitions, in words

The theorem speaks about five kinds of object. Each is defined in the file before the theorem. Appendix B.1 gives every definition with its line numbers; here is what each one means.

Formulas and the count (lines 13–51). A formula is a finite set of clauses over n variables, and a clause is a finite set of signed variables. The formula `F` has width at most k when every clause has at most k members (line 38). The number `count F` is how many of the 2^n assignments make the formula true (line 47). Because clauses and formulas are sets, a repeated clause collapses into one; the empty clause is allowed and can never be satisfied; and a clause holding both a variable and its negation is allowed and counts as two members. The manuscript says the same thing in its own words: "Repeating a literal permits clauses with fewer than k literals" (p. 1, line 44).

The input encoding (lines 53–81). The machine does not read a formula; it reads a string of bits. The file fixes how a formula becomes bits. Numbers are written in unary, that is, n is written as n ones and a zero. Each clause becomes a sorted list of such numbers, the formula becomes a list of clauses, and the number of variables is written in front. Two things matter. First, the number of variables is part of the input, so a variable used in no clause still counts. Second, the string is not long: its length is a polynomial in n and m, which means it grows like a fixed power of them and not like 2^n. A formula with m clauses of width k on n variables takes at most about $8\,k\,n\,m$ bits. In symbols, with L the length of the string,

$$L \;\le\; n + 2 + m\,\big(2k(4n+1)+4\big) \;=\; O(k\,n\,m),$$

where $O(k\,n\,m)$ is read "at most a fixed multiple of $k\,n\,m$". A formula with 125 variables, 500 clauses and width 4 becomes 1,006,223 bits; one with 1,000 variables, 4,000 clauses and width 3 becomes 47.9 million bits. Both numbers are from a script that imitates the definitions, `reports/scratch/eth_audit/check_encoding.py`. Nothing in the encoding grows exponentially.

The machine (lines 83–128). The machine is a Turing machine, the textbook model of a computer: a finite control that reads and writes symbols on tapes, one cell at a time. This one has several tapes: a finite table of states, a finite number of tapes, and a rule that, in each state, reads one cell of one tape and then either stops with a yes-or-no answer, or writes one symbol, or moves

one head by one cell. Every part of it is finite. A step is one lookup in a table with three rows per state. No part of the machine can hide an unbounded computation inside a step. The tapes and the moves are Mathlib's standard ones (Mathlib is the community library of mathematics for Lean, at the revision the collection pins, `d13f23b7`).

Runs (lines 87–90). A *configuration* is a snapshot: the control's current state and the contents of every tape. A *trace* of length t is a record that the machine, started in one configuration, reached another in exactly t successful steps. The step rule is a function, so each run is determined by its input; there is no randomness and no guessing.

Space monitoring (lines 147–189). Beside each configuration the file keeps, for every tape, the furthest cells the head has reached to the left and to the right. The *visited space* is the total number of cells ever visited, summed over the tapes. The *count word* is a number written in binary, least significant bit first (line 147).

The exponent and the bound (lines 130–141). The theorem's running-time bound is built from one function of n, which the file calls `runtimeScale` and we write $\rho(n)$:

$$\rho(n) \;=\; \frac{n\,\bigl(\ln\ln(n+16)\bigr)^2}{\sqrt{\ln(n+16)}},$$

with ln the natural logarithm. In words: take n; multiply it by the square of the logarithm of the logarithm of $n+16$; divide by the square root of the logarithm of $n+16$. The manuscript uses the same logarithm: "Throughout, log denotes the natural logarithm" (p. 1, lines 48–49; again at p. 7, lines 326–327). The $+16$ keeps the double logarithm defined and positive at every n, including $n = 0$, so the theorem needs no condition on n. The bound itself, called `runtimeBound` (lines 136–139), is

$$C\,(L+1)^d\,2^{A\,\rho(n)},$$

that is, a constant C, times a power d of the input length L, times two raised to A times $\rho(n)$. We call $\rho(n)$ *the exponent* and $C(L+1)^d\,2^{A\rho(n)}$ *the bound* throughout the paper. The constants C, A and d are the subject of the next subsection.

2.3 What the theorem says, and in what order

Lines 194–217 state the theorem. Appendix B.2 displays it with every quantifier and the line of every clause. In words, for every fixed width k of at least 2:

> There exist a machine, a count tape, an answer tape, and five positive constants C, A, S, d, e, such that for every number of variables n and every formula F on n variables of width at most k, the machine started on the encoding of F halts after some number of steps, `time`, with the count of satisfying assignments written in binary on the count tape, the bit "is the count positive" on the answer tape and in its halting state, a count word of at most $n+1$ bits, `time` at most $C\,(L+1)^d\,2^{A\rho(n)}$, and never more than $S\,(L+1)^e$ cells visited.

Three features of the order of the words matter for everything that follows.

The machine and the constants come after k and before n. For each width k there is one machine and one set of constants, and they serve every formula of every size. They may depend on k and on nothing else. This is what complexity theorists write as $O_k(1)$: a constant that depends only on k.

There is no lower limit on n. The manuscript's Theorem 1.1 and its Proposition 6.4 are stated for $n \geq 20$ (p. 2, lines 56–57; p. 43, lines 2474–2475). The verified theorem has no such condition; the $+16$ in the exponent is what makes that possible.

The constants exist, and that is all. The theorem says that values of C, A, S, d, e exist. It names none of them. The proof obtains them from other existence lemmas (a lemma is a small theorem proved on the way to a larger one) and never writes a number down. We read the proof for the values it does pin: at the point where the main run is assembled, the exponent constant is set to $A = 256 + A_t$ with A_t at least 1 (`SqrtLogSpace/NativeLargeEntry.lean`, line 51; `SqrtLogSpace/NativeLeafContract.lean`, lines 792–793). So the proof as written has $A \geq 257$, and the theorem that the comparator exhibits adds one more (Section 4, $A \geq 258$). No closed value of any of the five constants exists anywhere. The machine is explicit in one respect: it is produced mechanically ("compiled") from a program that uses 80 one-bit memory cells ("registers") and 1,220 tapes (`SqrtLogSpace/Final.lean`, lines 98–101). How many states it has is never computed.

2.4 The exponent, in numbers

Everything the title says rests on one comparison: the exponent $\rho(n)$ against n itself. If $\rho(n)$ is below n, the bound's exponential part $2^{\rho(n)}$ is below 2^n, the cost of trying every assignment. If $\rho(n)$ is above n, it is not.

The ratio $\rho(n)/n$ depends only on $y = \ln(n + 16)$: it equals $(\ln y)^2/\sqrt{y}$. That ratio starts at 0.64 for $n = 1$ and rises. It crosses 1 between $n = 49$ and $n = 50$: $\rho(49)/49 = 0.9994$ and $\rho(50)/50 = 1.0027$. It keeps rising to a peak of $16/e^2 = 2.165$, reached when $\ln y = 4$, which is at $n \approx 10^{23.7}$, a number with 24 digits. Then it falls, and it is back below 1 only when $y = 5503.7$, which is at $n \approx e^{5504} \approx 10^{2390}$, a number with 2,391 digits. Table 2.1 gives the values and Figures 2.1 and 2.2 draw them. Every number is printed by `paper3/scripts/runtime_scale.py`, run under `python3 -I`; it imports nothing from the clone and works in y, so that no n beyond the range of floating-point arithmetic is ever formed.

Two facts are true at once, and the title is about the second. In the limit, $\rho(n)/n$ goes to 0: the exponent grows more slowly than n, which is what "subexponential" means, and that is a theorem in our companion Lean (Appendix A, A.6). And on every size anyone will ever write down, from 50 to a number with 2,390 digits, the exponent is above n.

2.5 The title's claim, made precise

The title says the verified algorithm is slower than trying all 2^n assignments at every practical size. Read against the theorem, that means the following four things, and nothing more.

1. **What is verified is a guarantee, not a running time.** The theorem says the machine halts within $C(L+1)^d\, 2^{A\rho(n)}$ steps. It does not say how many steps the machine actually takes

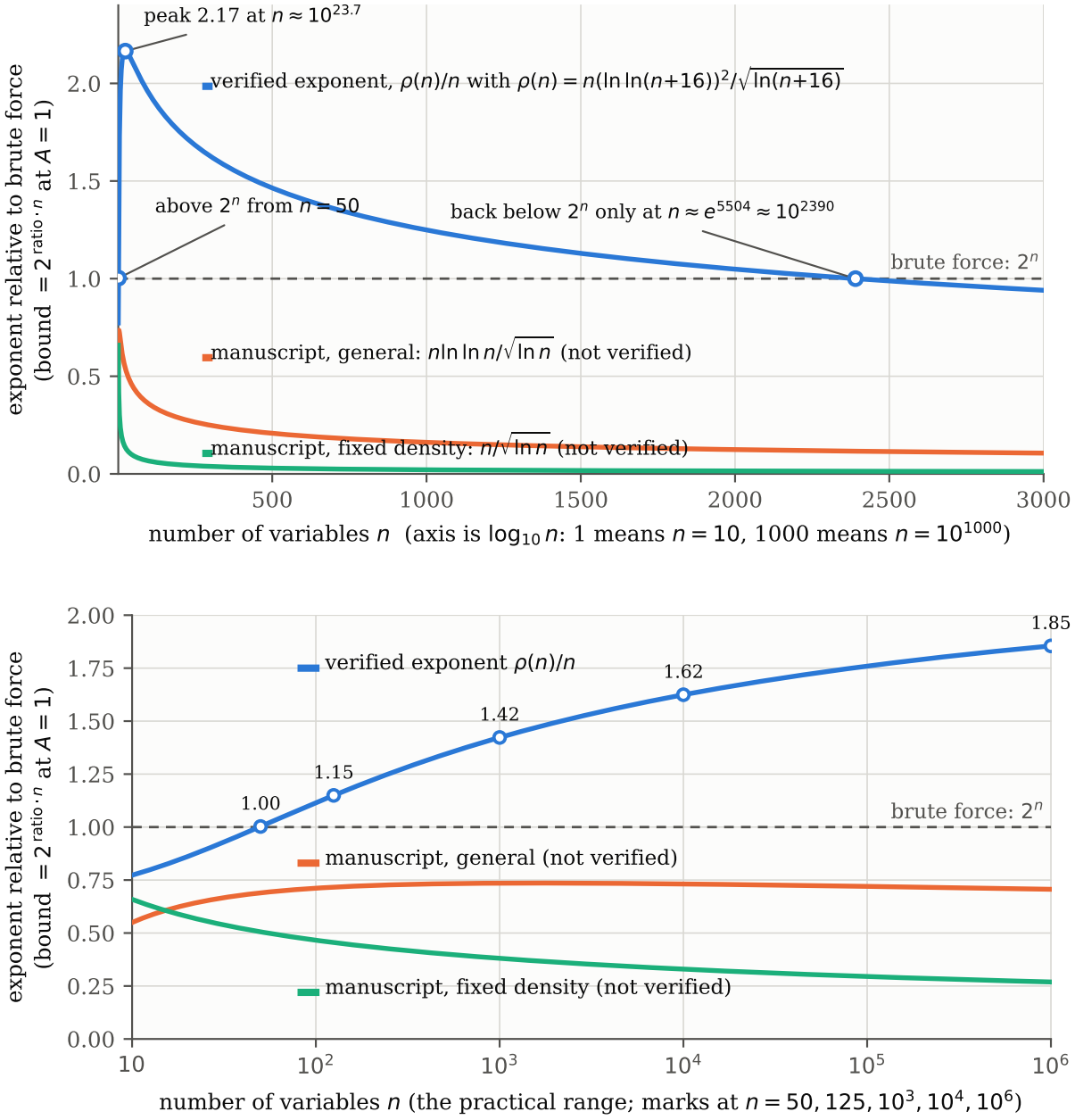


Figure 2.1: The verified exponent against trying every assignment. Each curve is an exponent divided by n; the dashed line at 1 is 2^n, the cost of trying every assignment, and a curve above the line is a guarantee worse than that. Top: the whole range, the horizontal axis being the number of digits of n, from $n = 10$ to $n = 10^{3000}$. The solid curve is the verified exponent $\rho(n)/n$; the two curves beneath it are the manuscript's two sharper exponents, $n \ln\ln n/\sqrt{\ln n}$ (the manuscript's Theorem 1.1, general case) and $n/\sqrt{\ln n}$ (its fixed-clause-density case), which are not in the Lean library (Section 3). Bottom: the same three curves over the practical range $n = 10$ to 10^6, with $\rho(n)/n$ marked at $n = 50, 125, 10^3, 10^4, 10^6$. All curves take the constant A to be 1; the verified A is unknown and at least 257 in the proof on record. Regenerate: `python3 -I paper3/scripts/rho_over_n.py`.

Table 2.1: The verified exponent compared with n. The second column is the ratio $\rho(n)/n$; where it is above 1, the bound's exponential part $2^{\rho(n)}$ (with $A = 1$) is above 2^n, the cost of trying every assignment. The third column is $\rho(n)$ itself. $\rho(n) = n(\ln\ln(n+16))^2/\sqrt{\ln(n+16)}$, the comparator's `runtimeScale` (lines 132–134), natural logarithms. Regenerate: `python3 -I paper3/scripts/runtime_scale.py`.

n	$\rho(n)/n$	$\rho(n)$
1	0.6443	0.64
20	0.8606	17.21
49	0.9994	48.97
50	1.0027	50.14
100	1.1146	111.46
125	1.1495	143.69
10^3	1.4229	1422.88
10^6	1.8550	1.85×10^6
10^{12}	2.0956	2.10×10^{12}
$10^{23.7}$	2.1654	—
10^{100}	1.9497	—
10^{1000}	1.2490	—
$e^{5504} \approx 10^{2390}$	1.0000	—
10^{3000}	0.9403	—

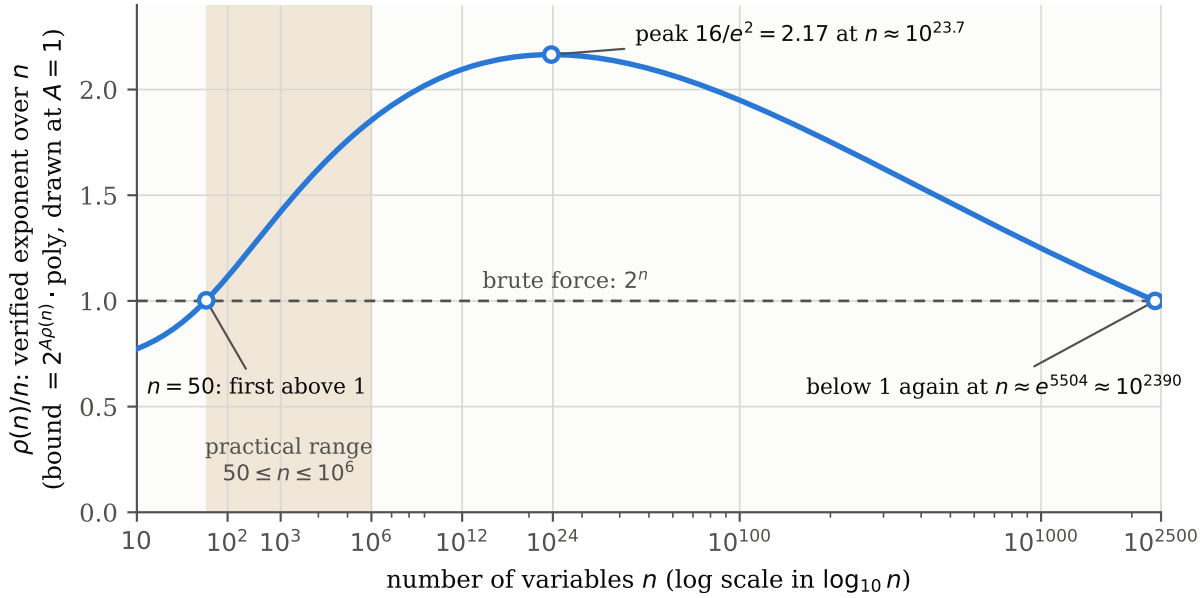


Figure 2.2: The ratio $\rho(n)/n$ alone, over an enormous range: the horizontal axis is the number of digits of n, from $n = 10$ to $n = 10^{2500}$. The level 1 is the cost of trying every assignment. Marked: the first whole number above the level ($n = 50$); the peak ($16/e^2 = 2.17$, at $n \approx 10^{23.7}$); the return below the level ($n \approx e^{5504} \approx 10^{2390}$); and, shaded, the practical range $50 \le n \le 10^6$. Regenerate: `python3 -I paper3/scripts/runtime_scale.py`.

on any input. No implementation of the machine exists to be timed. A bound above 2^n does not say the machine is slow; it says the proof promises nothing better than 2^n there.

2. **The constants are never computed.** The theorem asserts that C, A, S, d, e exist. The proof builds them from other existence lemmas and states no value, beyond the $A \geq 257$ read above. The machine's number of states is likewise never computed.

3. **With $A = 1$, the verified exponent is above n for every n from 50 up to about 10^{2390}.** That is Table 2.1. Over that range the exponential part of the bound is at least 2^n, and at its worst it is $2^{2.17\,n}$. With the true A, whatever it is, the exponential part is $2^{A\rho(n)}$, which is at least $2^{\rho(n)}$ as soon as $A \geq 1$; and the proof's own construction has $A \geq 257$. So nothing on record puts the bound below 2^n at any practical size.

4. **Hence, at every practical size, the verified guarantee is weaker than the guarantee of trying all assignments.** Trying all 2^n assignments and checking each against the m clauses counts the satisfying ones in 2^n times a small polynomial in $n + m$ steps, with constants one can write down. The verified guarantee is $C(L+1)^d\, 2^{A\rho(n)}$ with $\rho(n) > n$ on $50 \leq n < e^{5504}$ and C, A, d unknown. For every $C \geq 1$, $A \geq 1$ and $d \geq 1$ the verified bound is above 2^n on every n in that range. The only values on record are the proof's own, which has $1 \leq C$ and $1 \leq A$ in the statement of its main run (`SqrtLogSpace/NativeLargeEntry.lean`, line 12) and $A \geq 257$ in its construction. A proof with $A < 1$ would be a different proof; none is on record.

None of this is a defect of the formalisation or of the proof. It is what a theorem of this shape means when the constants in it are not tracked. The manuscript itself calls $\rho(n)$ "the weaker exponent" (p. 2, line 63), and its sharper exponents are not in the Lean library (Section 3). One detail of bookkeeping: inside the solution the exponent is written with base-2 logarithms, and a lemma shows the two forms differ only by a fixed multiplier, which can be folded into the constant A (Appendix B.2); nothing above depends on it.

2.6 What the statement does establish

For each fixed width $k \geq 2$, there is a deterministic machine that, given any k-CNF on n variables with m clauses in the fixed encoding, halts with the exact number of satisfying assignments written in binary on a named tape, within $C(L+1)^d\, 2^{A\rho(n)}$ steps. Here L grows no faster than a polynomial in $n + m$, and $\rho(n)$ grows more slowly than n. A formula of width k on n variables cannot have more than about n^k distinct clauses (2,604,376 at $n = 125$, $k = 3$; Appendix B.2), so the whole bound is subexponential in n alone. In the standard vocabulary, where $o(n)$ means a quantity that grows more slowly than n and poly a polynomial: a deterministic $2^{o(n)} \cdot \mathrm{poly}(n + m)$ algorithm for counting the solutions of k-SAT, and therefore for deciding it. That refutes the Exponential Time Hypothesis (Impagliazzo and Paturi, 2001), the conjecture that 3-SAT needs time exponential in n, as stated. The verified theorem is the manuscript's own verified form, the weaker exponent of its Proposition 6.4 (p. 43, lines 2474–2476), without the $n \geq 20$ condition. We found no field, definition or quantifier through which the statement could be accepted by the kernel and yet mean something weaker than this; the audit behind that sentence is `reports/scratch/eth_audit/statement_audit.md`. Our own restatement of the theorem as a named Lean sentence, over a verbatim copy of the machine model, is (Appendix A, A.2); it is never proved and never assumed there (Section 8).

2.7 What this does not claim

- **No running time of any machine.** Nothing here measures or estimates how long the verified machine, or any program, takes on any input. The title compares two *guarantees*. Section 7 is the only place in the paper where anything is timed, and it times ordinary solvers, not this machine.
- **No statement about the proof.** That the statement means what we say it means is a reading of 220 lines. Whether the proof is sound is Section 4 and Section 5, which found no flaw. The one proof-side fact quoted here, $A = 256 + A_t$, is a line of the solution read for the value it assigns, not a claim about the argument.
- **No value of the constants.** $A \geq 257$ is what the proof writes down, not what the machine needs; a different proof of the same statement could have any $A > 0$. The statement itself fixes nothing.
- **Nothing about the manuscript's sharper exponents**, which are not in the Lean library and are Section 3's subject.
- **No claim about sizes beyond e^{5504}.** Past $n \approx 10^{2390}$ the verified exponent is below n again at $A = 1$; we say nothing about that range except that it is not a practical one.

3 What the manuscript claims beyond the verified statement

The manuscript (OpenAI, 2026) is 67 pages long. The verified statement is 220 lines long. Section 2 read the statement. This section reads the manuscript's claims one by one. For each claim it says one of three things: this is what the Lean theorem asserts; this follows from the Lean theorem by an argument that exists only on paper; or this rests on a bound that the Lean library does not contain at all.

The collection draws the line itself, in a short note on the family. We quote its two sentences in full (`lean/docs/378.md`, lines 9 and 11):

> "For fixed $k \geq 2$, Lean proves that a deterministic machine exactly counts satisfying assignments of a canonically encoded k-CNF on n variables in time $(L+1)^{O_k(1)}\, 2^{O_k(n(\log\log n)^2/\sqrt{\log n})}$, where L is the encoding's bit length. At every step, the machine has visited at most $(L+1)^{O_k(1)}$ tape cells in total."
>
> "For m clauses, the October 25 manuscript gives the sharper time bound $(n+m)^{O_k(1)}\, 2^{O_k(n\log\log n/\sqrt{\log n})}$. When $m \leq Dn$ for fixed $D > 0$, this improves to $(n+m)^{O_{k,D}(1)}\, 2^{O_{k,D}(n/\sqrt{\log n})}$."

In words (the notation $O_k(1)$ means a constant that depends only on k, Section 2): the Lean proves the bound with the exponent of Section 2, the one with the *squared* double logarithm. The manuscript claims two better exponents, one with the double logarithm not squared, and one, for formulas with few clauses, with no double logarithm at all. Everything below is an elaboration of that note against the manuscript's text and the library's files. Table 3.2 is the summary.

3.1 Three statuses, and what "paper-only" means

We tag every claim with one of three words.

- **Verified.** The claim is the comparator statement `exact_count_and_sat`, or one of its parts, or a theorem of the solution tree that the replay of Section 5 compiled and whose axioms it printed. Nothing else is called verified.
- **Supported.** The claim follows from the verified statement by an argument that is on paper only: a short reduction, or a citation. The argument is not in the Lean library, and we have not checked it. What we have checked is narrower: the only property of the counter the argument uses is one the verified statement provides, and the exponent it inherits is therefore $\rho(n)$ and not a sharper one.
- **Paper-only.** The claim rests on a bound that the Lean library at commit `4eaee875` does not contain: Theorem 1.1's two sharper exponents, and whatever is derived from them.

Our working plan for this section had two tags, verified and paper-only. The middle one is ours. The manuscript's corollaries are of two different kinds, and one word would have put Corollary 1.3, an encoding into the verified counter, beside Corollary 1.4, a bound whose proof calls the sharper exponent by name. In that two-way split, "supported" is a kind of "paper-only": not in the library. In no case does "paper-only" mean "wrong". We did not read the manuscript's proofs for correctness. The tag records where a claim is established, not whether it is.

3.2 The sharper exponents of Theorem 1.1

Theorem 1.1 (p. 2, lines 56–62) states two running-time bounds for the same algorithm. The first holds for every k-CNF on $n \geq 20$ variables: a polynomial in $n + m$ times $2^{O_k(n \log\log n/\sqrt{\log n})}$. The second holds when the number of clauses is at most a fixed multiple D of n: a polynomial times $2^{O_{k,D}(n/\sqrt{\log n})}$. Both use a polynomial amount of space (line 60). The sentence that follows the theorem is the one this paper's title is about (p. 2, lines 63–67):

> "The rational interpolation argument in Section 6.4 gives the weaker exponent $O_k(\rho(n))$, where $\rho(n) = n(\log\log n)^2/\sqrt{\log n} = n/(\log n)^{1/2+o(1)}$. Appendix D proves both bounds in Theorem 1.1 by tensor contraction."

Proposition 6.4 (p. 43, lines 2474–2476) states the weaker bound on its own: for each fixed $k \geq 2$, the interpolation algorithm counts in a polynomial times $2^{O_k(\rho(n))}$ bit operations and polynomial space, for $n \geq 20$.

Which of the three is in the Lean. The verified exponent is the weaker one. The comparator's `runtimeScale` (lines 132–134 of the comparator file) is $n(\ln\ln(n+16))^2/\sqrt{\ln(n+16)}$, the squared form. The solution's own exponent, called `rho` (`SqrtLogSpace/Asymptotics.lean`, lines 10–11), is the same function with base-2 logarithms, and a lemma, `runtimeScale_le_four_rho` (lines 190–191), bounds the comparator's exponent by four times it for $n \geq 20$. The proof is built around the parameters of the manuscript's Section 6.4, the three numbers that set how the proof thins the formula before counting (its "sparsification"), to the letter: the manuscript selects $\lambda_n = \lceil \log_2 n \rceil$, $\nu_n = \lceil \log_2 \lambda_n \rceil$ and $s = \lfloor \sqrt{\lambda_n}/\nu_n^2 \rfloor$, "the largest nonnegative

integer satisfying $s^2\nu_n^4 \le \lambda_n$" (p. 43, lines 2480–2486; $\lceil x \rceil$ is the smallest whole number at least x and $\lfloor x \rfloor$ the largest at most x), and `Asymptotics.lean` lines 14–20 define `sparsifyLambda`, `sparsifyNu` and `sparsifySchedule` as exactly those three quantities. What Appendix D of the manuscript proves is not in the library in any form we could find (Section 3.6). So the general exponent $n \log\log n/\sqrt{\log n}$ and the fixed-density exponent $n/\sqrt{\log n}$ are **paper-only**: proved at p. 57 (lines 3222–3246) and p. 54 (lines 3045–3064) respectively, by the contraction theorem of Appendix D.

Both bounds from one algorithm. The manuscript notes that a single algorithm attains both paper-only bounds, by switching between two methods according to the number of clauses (p. 57, lines 3247–3250). That algorithm is not the verified one either.

The three exponents, in numbers. Table 3.1 puts the three exponents side by side, each divided by n and each with its constant taken to be 1. A value above 1 means the exponent is above n, so the bound is worse than trying every assignment. The verified ratio is the one of Table 2.1. The general paper-only ratio, $\ln\ln n/\sqrt{\ln n}$, never reaches 1: it peaks at $2/e = 0.736$ at $n \approx 1{,}618$ and falls on both sides. The fixed-density ratio $1/\sqrt{\ln n}$ falls from 0.578 at $n = 20$. So, at constant 1, both paper-only guarantees sit below 2^n at every size, and the verified one sits above 2^n from $n = 50$ to $n \approx 10^{2390}$. That is the whole content of the word "verified" in the title: the manuscript's exponents would make a different sentence, and they are not what the kernel checked. Two cautions apply to the table equally. The constants in front of every exponent are not computed anywhere, in the manuscript or in the Lean. And the manuscript's exponents are undefined below $n = e$ and are stated for $n \ge 20$, while the verified one is stated for all n. Figure 2.1 draws the same three curves.

Table 3.1: The three exponents of the manuscript, each divided by n, with every constant taken to be 1. A value above 1 means the bound is worse than trying every assignment at that n. Columns: the verified $\rho(n)/n = (\ln\ln(n+16))^2/\sqrt{\ln(n+16)}$ (comparator lines 132–134); Theorem 1.1's general exponent $\ln\ln n/\sqrt{\ln n}$ (p. 2, line 59); its fixed-density exponent $1/\sqrt{\ln n}$ (p. 2, line 62). Natural logarithms throughout (p. 1, lines 48–49). The row e^{e^2} is where the general ratio peaks ($2/e$). Regenerate: `python3 -I paper3/scripts/beyond_claims.py`.

n	verified $\rho(n)/n$	general $\ln\ln n/\sqrt{\ln n}$	density $1/\sqrt{\ln n}$
20	0.861	0.634	0.578
50	1.003	0.690	0.506
125	1.150	0.717	0.455
10^3	1.423	0.735	0.380
$e^{e^2} \approx 1{,}618$	1.472	0.736	0.368
10^6	1.855	0.706	0.269
$10^{23.7}$	2.165	0.541	0.135

The abstract's form. The manuscript's abstract states the running time as a polynomial times $2^{n/(\log n)^{1/2+o(1)}}$ (p. 1, lines 6–9). The small term $o(1)$ in that exponent is a quantity that goes to zero as n grows. The verified exponent is of that form too, because the square of a double logarithm grows more slowly than any fixed power of the logarithm (in symbols, $(\ln\ln n)^2$ is $(\ln n)^{o(1)}$), and

the manuscript says so at lines 64–66. We tag the abstract's form **verified**, with the remark that its $o(1)$ hides exactly the factor by which the three rows of Table 3.1 differ: at $n = 10^6$ that factor is 6.9 for the verified exponent, 2.6 for the general one and 1 for the fixed-density one.

3.3 The corollaries

The corollaries speak of objects from the manuscript's own fields: circuits, graphs, maps. We gloss each in a clause where it first appears and define none further; a reader who needs more should go to the manuscript at the lines cited.

Finding and sampling a satisfying assignment (p. 2, lines 68–71). "When $\#F > 0$, the standard self-reduction finds a satisfying assignment by fixing one variable at a time and checking that the remaining count is positive. Choosing each bit according to the two conditional counts instead gives an exactly uniform random satisfying assignment," with polynomial expected overhead and a citation. **Supported**. The method calls the verified counter at most n times on formulas of the same width, and the verified statement provides the exact count it needs. The method is two sentences of prose and is not in the library.

Corollary 1.2, Max-k-SAT within εm (p. 2, lines 77–94). A randomised algorithm finds an assignment that satisfies at least $\mathrm{OPT}(F) - \varepsilon m$ clauses, that is, within a small fraction of the best possible, with high probability, in time

$$(n+m)^{O_{k,\varepsilon}(1)}\, 2^{O_{k,\varepsilon}(\rho(n))}$$

(line 80). The corollary is stated with $\rho(n)$ itself, the verified exponent. Its proof samples $O(n/\varepsilon^2)$ clauses, builds a small circuit (a network of logic gates), turns the circuit into a 3-CNF with $O_{k,\varepsilon}(n)$ variables by a cited lemma, and calls Theorem 1.1 repeatedly, narrowing down the best value and then fixing variables one at a time (lines 82–94). **Supported**: the exponent matches the verified one because $\rho(cn) = O_c(\rho(n))$, and every call is to a fixed-width counter on $O(n)$ variables; the sampling, the circuit and the reduction are on paper. Its randomness is in the reduction, not in the counter.

Corollary 1.3, independent sets, colourings, perfect matchings (p. 2–3, lines 100–120). For a graph on v vertices and e edges: the number of independent sets (sets of vertices no two of which are joined by an edge) and, for fixed q, of proper q-colourings (colourings with q colours such that no edge joins two vertices of the same colour), in time $2^{o(v)}$, and the number of perfect matchings (sets of edges touching every vertex exactly once) in time $2^{o(v+e)}$, all in polynomial space. The encodings are explicit: one variable per vertex and a clause of two literals per edge (lines 107–112); q variables per vertex with "exactly one colour" and "no equal colours" clauses, a q-CNF on qv variables (lines 112–115); one variable per edge and a linear-size circuit for "exactly one selected edge meets each vertex", reduced to 3-CNF with $O(v+e)$ variables, with the extra variables fixed by the edge bits so that the count is preserved (lines 116–119). **Supported**: the conclusion is $2^{o(\cdot)}$, which needs only that $\rho(n)$ grows more slowly than n (Appendix A, A.6), and each encoding preserves the count into a fixed-width CNF on $O(v)$ or $O(v+e)$ variables. None of the three encodings is in the library.

Corollary 1.4, a circuit lower bound for E^{NP} (p. 3, lines 121–127). "The fixed-density case of Theorem 1.1 gives a circuit lower bound for E^{NP}" (line 121). A circuit lower bound is a proof that some problem needs large circuits; E^{NP} is a class of problems whose definition is not needed here. The claim: there is a problem L in that class and a constant $c > 0$ such that L needs circuits of more than $c\,n\sqrt{\log n}$ gates for infinitely many n (lines 123–127). The proof is Appendix C.1 (p. 51–52, lines 2925–2967), an argument of Williams's, and it invokes the paper-only exponent in so many words: "Put $a(t) = \sqrt{\log_2 t}$. Testing whether the count in the constant-density part of Theorem 1.1 is positive decides 3-SAT with V variables and at most $5V$ clauses in time $V^b\,2^{AV/a(V)}$ for fixed $A, b > 0$" (p. 52, lines 2949–2950); the threshold $s_n = \lfloor \eta n \lfloor a(n) \rfloor \rfloor$ is chosen against that a (line 2953), and the constant c comes out as $\sqrt{\eta/(d \log 2)}$ (line 2967). **Paper-only**: the quantitative bound $c\,n\sqrt{\log n}$ is tied to $2^{AV/\sqrt{\log V}}$, which the library does not contain. What the same argument would give from the verified exponent is a question about the manuscript's proof, which we do not examine (Section 3.8).

Corollary C.1, the class E^{NP} has no linear-size circuits (p. 51, lines 2912–2916). "Williams proved that if ETH is false, then E^{NP} does not have linear-size circuits [Wil13, Theorem 6.1]. Apply this implication to the deterministic subexponential-time 3-SAT algorithm supplied by Theorem 1.1." **Supported**: the only input is a deterministic $2^{o(n)}$ 3-SAT decision, which the verified statement provides on its answer tape for $k = 3$; the implication is a citation. The manuscript's own scope sentence applies: the corollary "does not assert a superpolynomial circuit lower bound for NP" (lines 2923–2924).

Corollary C.2, inverting constant-locality maps (p. 52, lines 2974–2977). For a Boolean map whose every output bit reads at most d input bits (a "constant-locality" map), the number of inputs mapping to a given output, and one such input, in deterministic time $(n+m)^{O_d(1)}\,2^{O_d(\rho(n))}$ and polynomial space, for $n \geq 20$. Stated with $\rho(n)$. The proof writes one clause per forbidden local assignment, a CNF of width at most d on the original n variables with at most $2^d m$ clauses whose satisfying assignments are exactly the wanted inputs, and applies Theorem 1.1 with $k = \max\{2, d\}$ (lines 2979–2988). **Supported**: the encoding is on paper; the counter and the exponent are the verified ones. The remark that follows, on Goldreich's candidate one-way functions (functions easy to compute and conjectured hard to invert; p. 53, lines 2994–2999), inherits the same status and the manuscript's own limit, "This does not rule out one-wayness against polynomial-time algorithms" (line 2999).

3.4 Appendix D: the contraction route

Appendix D of the manuscript, "A sharper bound for exact counting" (p. 53–64, from line 3001), proves Theorem 1.1 "by contracting Boolean tensor networks. The two running times reflect the number of wires in the networks" (lines 3002–3003). A tensor network is a way of writing one large sum as a product of small tables; the terms in the rest of this paragraph are the manuscript's, and we do not define them further. Theorem D.1 (p. 53, lines 3017–3028) is the contraction theorem: a bipartite tensor network with at most Cs edges, maximum degree at most $\lceil s^{9/10} \rceil$ and entries in $(\mathbb{Z}/2^{P_0}\mathbb{Z})[V]/(V^\ell)$ is contracted in $(s+L)^{O(1)}\,2^{O(s/\sqrt{\log s})}$ bit operations and polynomial space. The fixed-density proof (p. 54, lines 3045–3064) encodes a formula as a network with $O_k(n+m)$ edges and applies Theorem D.1 with $s = n + m$; the arbitrary-density

proof (p. 57, lines 3222–3246) combines the sparsification of Section 6 with Lemma D.2 (p. 54, line 3069) and reaches the exponent $O_k(n \log\log n / \sqrt{\log n})$ at line 3243. Section D.4 changes the logarithm convention for its own proof, "log denotes a base-two logarithm and ln denotes a natural logarithm" (p. 57, lines 3251–3252). All of this is **paper-only**: no declaration about tensor networks, contraction, Hermite recovery or packets exists in the family's 3,977 files, by the identifier search of Section 3.6. We make no statement about Appendix D's argument; we did not read it for correctness.

3.5 The hypotheses of §1.1.1 and the fine-grained remarks of §1.1.3

§1.1.1 (p. 4, lines 187–191). "Theorem 1.1 refutes ETH, SETH, Gap-ETH, and NSETH. Indeed, testing whether the exact count is positive or zero decides both satisfiability and unsatisfiability in deterministic $2^{o(n)}(n+m)^{O_k(1)}$ time for every fixed k; deciding satisfiability also solves the Gap-ETH promise problem. The deterministic algorithm also refutes the bounded-error randomized versions of ETH [...] and SETH [...], and of Gap-ETH." And §1.1.2 (p. 5, lines 211–212): "Our algorithm (Theorem 1.1) solves #3-SAT deterministically in $2^{o(n)}\,\mathrm{poly}(n,m)$ time and therefore refutes #ETH." These are the four named hypotheses about how hard satisfiability is, their randomised forms, and the counting hypothesis; Section 6 states each one in words. **Supported**, and by the shortest argument in the manuscript: the verified statement is a deterministic machine whose answer tape says whether the count is positive and whose count tape carries the count, within a subexponential bound, for each fixed $k \geq 2$; each hypothesis is the denial of that. The hypotheses themselves are not defined in the library (the strings "ETH" and "SETH" occur in the collection's Lean only in two files of a different family, `lean/OAI/Combinatorics/ParityLifts/`, which take an ETH-like proposition as a hypothesis and are not among the files the proof depends on, its closure, Section 4). Section 6 says exactly how the verified statement negates each one. The two sentences the manuscript adds at lines 191–194, on unconditional lower bounds for restricted proof systems and on Williams's 2016 MA-SETH protocol, are context, not claims about the result.

§1.1.3 (p. 5, lines 213–243). After a list of lower bounds proved from ETH or SETH (lines 213–225), the manuscript writes: "Our refutation of ETH and SETH removes one basis for the hardness claims above. Some of the same lower bounds follow from weaker hypotheses that may still hold" (lines 226–227). It names the Orthogonal Vectors Conjecture, a hypothesis about a different problem (lines 229–232), and the hypothesis that unrestricted CNF-SAT admits no $2^{(1-\delta)n}\,\mathrm{poly}(n,m)$ algorithm, adding: "Theorem 1.1 gives a running-time guarantee for each fixed clause width k, while this hypothesis allows the width to grow with n" (lines 235–236). These are **paper-only** remarks in the sense that nothing about any other hypothesis or any reduction is in the library; they are also the manuscript's own statement of what the result does not do, and Section 6 rests on them. The one positive transfer, Set Splitting with sets of size at most k by one variable per element and two clauses per set (lines 237–243), is **supported** in the same sense as Corollary 1.3: an encoding on paper into the verified counter, inheriting $\rho(n)$. The caveat at p. 2, lines 95–99, that $2^{o(n)}$ bounds "do not transfer under general polynomial-time reductions," is not a claim but a limit the manuscript places on its own result; it is quoted again in Section 6.

3.6 What the library carries beyond the comparator statement

"Paper-only" is defined against the whole library at commit `4eaee875`, not only against the comparator file, so we say what else the library proves about this family. The file `SqrtLogSpace/Final.lean` (201 lines) states five theorems, each followed by a `#print axioms` command, which asks Lean to list the axioms a proof uses (lines 197–201). The replay's build log carries the kernel's answer for all five: the three standard axioms (`paper3/replay/logs/run2/axioms_in_build.txt`).

- `compiled_exact_count_and_sat` (line 10) and `exact_count_and_sat` (line 74): the comparator statement, first over the explicit 80-register, 1,220-tape program and then over the machine it compiles to (Section 2).
- `exact_count_eth` (lines 104–147): the same conclusion with the polynomial factor written as $(n + |F| + 1)^d$ instead of $(L + 1)^d$, the exponent as $A \cdot \texttt{rho}\ n$ in base 2, and the condition $20 \leq n$ (line 109); its proof sets its own constants to twice, twice and four times the comparator theorem's d, e and A (line 132). This is Proposition 6.4's $(n+m)^{O_k(1)}\, 2^{O_k(\rho(n))}$ form in the library, so the manuscript's polynomial factor is verified as well as its exponent; it is **verified**, and it is still the weaker exponent.
- `sat_polynomial_space` (lines 149–183), for $k \geq 3$: the answer bit is true exactly when the formula is satisfiable (`answer = true ↔ SAT.Satisfiable F`), the decision form made explicit.
- `original_sat_main` (line 184): the sentence named `SqrtLog.Statement.MainTheorem`, a Lean `Prop` (Section 8; `SqrtLog/StatementCanonical.lean`, line 37), a decision-only target for every width $k \geq 3$ with the same `runtimeBound`, stated there without proof ("no inhabitant is supplied here") and proved in `Final.lean`.

Beyond these, a search for names over the family's 3,977 `.lean` files, by the read-only script `paper3/scripts/beyond_claims.py`, finds no occurrence of `SimpleGraph`, `PerfectMatching`, `Coloring`, `IndependentSet`, `CircuitSize`, `OrthogonalVectors`, `MaxSat`, `TensorNetwork`, `HermiteInterp`, `ENP`, `SIZE`, `SetSplitting`, `GapETH` or `NSETH` as an identifier: none of the corollaries, none of Appendix D, and none of the hypotheses is formalised. The comparator's JSON, `lean/ComparatorChallenges/SubexponentialSAT.json`, names one theorem, `exact_count_and_sat`, and no definition. A text search is necessary and not sufficient (Section 4); here it is used to show absence, where a miss would require a corollary to be formalised under none of the words its own statement uses.

3.7 The table

Table 3.2 lists every claim examined above with its status and the one-line reason.

3.8 What this does not claim

- **No judgement on the paper-only proofs.** Appendix D, Appendix C.1 and the two sharper bounds of Theorem 1.1 were located and quoted, not read for correctness. "Paper-only" means not in the Lean library at commit `4eaee875`; it does not mean wrong, and nothing here suggests it.

Table 3.2: Every claim of the manuscript examined in this section: where it is stated (page of `paper.pdf`, lines of the committed extract), its status in the sense of Section 3.1 (verified, supported, or paper-only), and the one-line reason. Generated, with every page and fragment checked against the extract and the clone, by `python3 -I paper3/scripts/beyond_claims.py`.

claim	stated	status	why
Abstract: time $(n+m)^{O_k(1)}\,2^{n/(\log n)^{1/2+o(1)}}$, polynomial space	p. 1, 6–11	verified (as a form)	$\rho(n) = n/(\log n)^{1/2+o(1)}$ too (p. 2, lines 64–66); the $o(1)$ absorbs the difference between all three exponents
Theorem 1.1, general: $(n+m)^{O_k(1)}\,2^{O_k(n \log\log n/\sqrt{\log n})}$	p. 2, 56–59	paper-only	proved in Appendix D.3 (p. 57, lines 3222–3246); the comparator has $(\log\log)^2$; `docs/378.md` line 11
Theorem 1.1, $m \le Dn$: $(n+m)^{O_{k,D}(1)}\,2^{O_{k,D}(n/\sqrt{\log n})}$	p. 2, 60–62	paper-only	proved in Appendix D.2 (p. 54, lines 3045–3064); no density case in the Lean; `docs/378.md` line 11
Theorem 1.1: $(n+m)^{O_k(1)}$ bits of space	p. 2, 60	verified	the conjunct `visitedSpace` $\le S(L+1)^e$ at every prefix, $L = O(knm)$; as $(n+\lvert F\rvert+1)^e$ in `Final.lean` 104–147
The weaker exponent $O_k(\rho(n))$, Proposition 6.4	p. 2, 63–66	verified	the comparator statement (Section 2), with the $n \ge 20$ proviso removed; Prop. 6.4 at p. 43, lines 2474–2476
Proposition 6.4's parameters $\lambda_n = \lceil \log_2 n \rceil$, $\nu_n = \lceil \log_2 \lambda_n \rceil$, $s = \lfloor \sqrt{\lambda_n}/\nu_n^2 \rfloor$	p. 43, 2480–2486	verified (the same parameters)	`Asymptotics.lean` lines 14–20 define `sparsifyLambda`, `sparsifyNu`, `sparsifySchedule` identically
"Appendix D proves both bounds in Theorem 1.1 by tensor contraction"	p. 2, 67	paper-only	no tensor-network declaration in the library (Section 3.6)
Self-reduction finds a satisfying assignment; exactly uniform sampling	p. 2, 68–71	supported	calls the verified counter n times; the reduction is prose; cites [JVV86]
Corollary 1.2: Max-k-SAT within εm, randomised, $2^{O_{k,\varepsilon}(\rho(n))}$	p. 2, 77–81	supported	stated with $\rho(n)$ itself; its proof (lines 82–94) samples clauses and reduces a threshold circuit to 3-CNF, on paper
Caveat: $2^{o(n)}$ bounds do not transfer under polynomial-time reductions	p. 2, 95–99	a caveat, not a claim	quoted in Section 6
Corollary 1.3: independent sets, q-colourings in $2^{o(v)}$; perfect matchings in $2^{o(v+e)}$	p. 2, 100–103	supported	counting-preserving encodings (lines 107–120), one variable per vertex, q per vertex, one per edge plus a circuit; needs only $\rho(n) = o(n)$
Corollary 1.4: $C_L(n) > c\,n\sqrt{\log n}$ for some $L \in \mathrm{E}^{\mathrm{NP}}$	p. 3, 123–127	paper-only	its proof (p. 51–52, lines 2925–2967) uses "the constant-density part of Theorem 1.1" (line 2949), the $2^{AV/\sqrt{\log_2 V}}$ bound
§1.1.1: Theorem 1.1 refutes ETH, SETH, Gap-ETH, NSETH and their bounded-error randomised forms	p. 4, 187–191	supported	the one-line reason is the manuscript's own: a positive-or-zero test on the verified count, in $2^{o(n)}(n+m)^{O_k(1)}$; Section 6
§1.1.2: Theorem 1.1 refutes #ETH	p. 5, 211–212	supported	the verified machine is a deterministic $2^{o(n)}\,\mathrm{poly}(n,m)$ #3-SAT counter
§1.1.3: "removes one basis for the hardness claims above"; OVC and unrestricted CNF-SAT may still hold	p. 5, 226–236	paper-only (remarks)	statements about other hypotheses and reductions; nothing of them is in the library; Section 6
§1.1.3: Set Splitting with sets of size $\le k$ in subexponential time	p. 5, 237–243	supported	one variable per element, two clauses per set (lines 239–243), on paper; inherits $\rho(n)$
Corollary C.1: $\mathrm{E}^{\mathrm{NP}} \not\subseteq \mathrm{SIZE}(O(n))$	p. 51, 2912–2916	supported	needs only a deterministic $2^{o(n)}$ 3-SAT decision (line 2915) plus Williams 2013, Theorem 6.1, cited
Corollary C.2: inversion and exact preimage counting of constant-locality maps, $2^{O_d(\rho(n))}$	p. 52, 2974–2977	supported	stated with $\rho(n)$ and $n \ge 20$; a width-d CNF on the same n variables, $k = \max\{2, d\}$ (lines 2984–2988)
Appendix D: Theorem D.1 (tensor contraction), Lemma D.2, both proofs of Theorem 1.1	p. 53, 3001–3028	paper-only	proofs at p. 54 (3045–3064) and p. 57 (3222–3250); D.4 uses base-2 logarithms (p. 57, lines 3251–3252)

- **No judgement on the supported arguments either.** The reductions behind Corollaries 1.2, 1.3, C.1, C.2 and the Set Splitting transfer are standard in kind and short, and we checked only which property of the counter they use. Whether each is correct as written is the manuscript's, not ours.
- **Nothing about what the verified exponent would yield through Corollary 1.4's argument.** A reader can rerun Appendix C.1 with $a(t)$ replaced by the verified rate; we did not, because it is a question about a paper-only proof.
- **No constants.** Every constant in this section is as unknown as the comparator's C, A, S, d, e; the comparisons of Table 3.1 are at constant 1 and say nothing about any machine's running time.
- **Nothing beyond the collection.** Whether the sharper bounds have since been formalised elsewhere, or the manuscript revised, is not examined; what is read is the commit and the dated manuscript.

4 The proof, inspected

Section 2 read the theorem. This section reads the proof, as far as reading can go. The question is not whether the mathematics is right. The kernel decides that, and Section 5 reports what the kernel said when we ran it. The question is whether anything in the source could make the kernel's acceptance mean less than it appears to mean. There are known ways for that to happen: an added axiom; a `sorry`, which is Lean's word for a proof left out; a step proved by trusting a compiled program rather than the kernel; a patched copy of the mathematics library; a statement in the solution that differs from the statement in the challenge; or a running-time bound proved against a cost model of the proof's own choosing. We found none of these. What we did find is that the repository offers no recorded evidence that the proof type-checks at all, that is, that Lean accepts it: no saved output of the axiom check, no automated build, and no second checker for this family. That is why Section 5 exists. The last subsection joins the two.

4.1 Method

The clone of `openai/math` at commit `4eaee875` is treated as untrusted data. Every file was read with `cat`, `sed`, `grep` and `python3 -I`, and nothing in it was built, imported or executed for this section. Two scripts from the audit of 2026-11-05 do the mechanical part. `closure.py` follows the `import` lines from the solution's main module, strips comments, and counts how often each keyword of interest occurs in what remains. `defdiff.py` compares each definition in the challenge file with the library's definition of the same name, ignoring differences in spacing. Both were re-run for this section, and every number below is from that fresh run; `paper3/scripts/proof_inspection.py` re-runs them, performs the remaining checks, and regenerates Table 4.1. The inspection reads the spine of the proof, that is, the final file, the compiler, the entry lemmas and the constants, and sweeps the whole closure by keyword. The arithmetic beneath the spine was not read (Section 4.9).

4.2 The closure

The *closure* of a module is the set of all files it depends on, directly or through other files. The comparator's JSON names one solution module, `OAI.Algorithms.SubexponentialSAT.Main`, a sixteen-line file that imports `SqrtLog.ExecutionNativeFinal` and `SqrtLogSpace.Final` and declares nothing (`Main.lean`). Its closure inside the repository is **3,091 modules**, every one in the family's own directory, totalling **550,551** lines; the largest file is 2,873 lines (`SqrtLogSpace/DisjointReuse/SparsePriorityProgram.lean`). Outside the family, the closure reaches **50** modules of Mathlib and one module of Lean's own library, `Lean.Elab.Tactic.Omega`, imported by `SqrtLogSpace/FiniteFallback.lean` (line 14) and `SqrtLogSpace/DisjointReuse/SparseCharging.lean` (line 5). That one module is a proof tactic, a tool for writing proofs; the closure defines no tactic of its own (Section 4.3). Nothing is imported from any other family of the collection or from any of the repository's other dependencies.

The family's directory holds **3,977** `.lean` files, so **886** are outside the closure. They are not reachable from the collection's root module either: `lean/OAI.lean` imports the family only at its line 49, and the build's default target is that root (`lean/lakefile.lean`, line 89, `@[default_target] lean_lib OAI`). They are never built by the default target and bear on nothing: dead files, not a hole. For completeness, a raw search of the 886 for `sorry`, `axiom`, `native_decide` and `admit` finds nothing, with or without comments.

4.3 The keyword sweep

Table 4.1 gives the counts over the 3,091 files with comments stripped. The ones that matter are the zeros, and each zero rules out one way of weakening a verified proof.

- No `axiom`, no `sorry`, no `admit`: nothing is assumed and no proof is left out.
- No `native_decide` and no `ofReduceBool`: nothing is proved by trusting a compiled program's answer instead of the kernel.
- No `implemented_by`, `extern`, `unsafe` or `partial`: no definition has a hidden implementation, and every recursion is one the kernel checks for termination.
- No `elab`, `macro`, `syntax` or `simproc`: no custom programming of Lean itself that could rewrite a goal outside the kernel's view.

The non-zero counts are ordinary. Nine declarations are `opaque`, a keyword that hides a definition's body from the proof-writing layer; every one of the nine has a body, and the kernel checks that body against its type, so nothing is assumed. The 1,048 `noncomputable` declarations, the eight `open Classical` and the four explicit uses each of `Classical.choice` and `propext` are the classical reasoning the comparator permits. The 148 instances and 40 local instances are standard bookkeeping declarations ("typeclass instances") that tell Lean, for example, that a type is finite or a property is decidable; they are harmless because such a declaration can only make a false claim by way of an axiom, and there is none. The only options set in the closure switch off a check on notation in two files and have no bearing on soundness. Appendix B.4 lists every one of these with its file and line.

Table 4.1: The inspection in numbers. Each row is one count over the 3,091 files of the closure, comments stripped where a keyword is counted; the script column says which script produced it. Regenerate: `python3 -I paper3/scripts/proof_inspection.py`.

quantity	value	script
modules in the closure of `Main.lean`	3,091	`closure.py`
raw lines in those modules	550,551	`closure.py`
external imports: Mathlib modules / core modules	50 / 1	`closure.py`
`.lean` files under the family / outside the closure	3,977 / 886	this script
`axiom`, `sorry`, `admit`, `native_decide`, `ofReduceBool`	0, 0, 0, 0, 0	`closure.py`
`implemented_by`, `extern`, `unsafe`, `partial`	0, 0, 0, 0	`closure.py`
`elab`, `macro`, `syntax`, `simproc`	0, 0, 0, 0	`closure.py`
`set_option` (both `quotPrecheck false`)	2	`closure.py`
`opaque` (all with bodies)	9 (9)	`closure.py`, this script
`noncomputable`	1,048	`closure.py`
explicit `Classical.choice` / `propext` / `open Classical`	4 / 4 / 8	`closure.py`
`instance` / `local instance`	148 / 40	`closure.py`
`termination_by` / `decreasing_by`	9 / 5	`closure.py`
statement definitions compared / identical / identical up to `noncomputable`	35 / 33 / 2	`defdiff.py`, this script
theorem statement, characters (normalised / raw), identical	1,124 / 1,353	this script
`#print axioms` commands: closure files / lines; family files / lines	8 / 19; 17 / 35	this script
recorded `depends on axioms` outputs in the clone	0	this script
challenge JSONs: total / `enable_nanoda` false / true / absent	418 / 413 / 2 / 3	this script
CI workflow files (`lock-prs.yml` only)	1	this script
manifest packages / patch files / patches for Mathlib	42 / 23 / 0	this script

4.4 The statement is the challenge's

The solution does not import the challenge file. Instead the library declares again, under the same names, every definition the statement uses, spread over seven files. So one has to check that the two copies agree. The definition diff compares **35** declarations: **33** are identical character for character once spacing is ignored, and the remaining two, `runtimeScale` and `runtimeBound`, differ only in that the challenge marks each definition `noncomputable` by itself where the library marks a whole section so (`SqrtLog/StatementScale.lean`, line 15), a difference the kernel does not see. The theorem statement itself, from its first word to the start of its proof, is **identical** in the two files, 1,353 characters raw and 1,124 after normalisation, at line 194 of the challenge and line 74 of `SqrtLogSpace/Final.lean`; the challenge's proof is the single word `sorry` at its line 218. Appendix B.3 names the seven files.

This textual match is a sanity check and not the guarantee. The comparator (Lean FRO, 2026) compares the two statements after exporting them from Lean, with every definition the statement mentions matched by name and content, so a library definition that differed from the challenge's under the same name would make the comparator fail rather than pass silently. What the text

confirms is that there is no such difference to find, and that the shape Section 2 read is the shape the library proves.

4.5 The spine of the proof

We followed the proof from its last file down to the lemmas that connect it to the machine of the statement. Appendix B.3 gives every name and line; here is what the spine shows.

The machine is a concrete table. The theorem's machine is compiled from a program over 80 one-bit registers and 1,220 Boolean stacks, with twelve kinds of instruction (`BitProgram.lean`, lines 10–26). The compiler (`SqrtLogSpace/GuardedCompiler.lean`, line 131) builds the machine's transition table case by case from the program and then renames the states to numbers by an appeal to a choice principle that names no particular numbering. The theorem only says such a machine exists, so this non-constructive step (one that proves something exists without producing it) costs it nothing; it is also the reason no running time can be read off the statement (Section 2).

The constants, as far as the proof pins them. For inputs above a cutoff, the main run supplies a trace within the bound with $1 \le C$ and $1 \le A$ (`SqrtLogSpace/NativeLargeEntry.lean`, line 12). Its exponent constant is `256 + At` (line 51), where `At` is at least 1 and comes from a contract on the leaf program (`SqrtLogSpace/NativeLeafContract.lean`, line 793). The cutoff itself is `max 20 (max Nleaf Ns)` (line 50), with `Ns` obtained from a statement that something holds for all large enough n, which yields no particular number, and never computed. Below the cutoff, a finite scanner gives `time` $\le 28700\,(L+1)^2$ (`SqrtLogSpace/NativeEntry.lean`, line 92). The two branches are folded together, and the final theorem's constants are those of the two branches added up: `C + 28700, A + 1, S + 5022741, d + 2, e + 2` (`SqrtLogSpace/Final.lean`, line 41). So the proof as written has an exponent constant of at least 258 in front of $\rho(n)$, a polynomial degree that depends on k, and a leading constant of at least 28,701. None of these is in the statement, which asserts only that they are positive, and none of the underlying quantities is computed anywhere. This confirms, from the proof side, what Section 2 said from the statement side: the constants are unknown, and the *smallest* they could be is already far from 1.

Where a hidden cost model would live, and why it cannot. A proof of a running-time bound for a machine that is compiled from a higher-level program could, in principle, prove the bound against a cost function of its own and leave the kernel to check only that function's arithmetic. That is not what happens here. The lemmas that connect the program to the machine conclude in the challenge's own words: a source run of n steps becomes a run of the compiled machine of $7n$ steps, with the overhead 7 proved as a theorem (`SqrtLogSpace/GuardedCompiler.lean`, lines 152–162), and the space lemma (`SqrtLogSpace/CompileSpace.lean`, lines 125–143) produces a trace of the challenge's own step function with its visited space bounded on every prefix. The `time` in the final theorem is literally the length of a trace of the challenge's step function (its line 201). No cost function of the compiler's appears in the statement; a wrong cost model would make these lemmas unprovable, not the theorem false. This is the structural point

on which the rest of the paper leans: whatever the proof's internal accounting, the number the theorem bounds is the number of steps of the comparator's machine.

Correctness of the count. The outer lemma (`SqrtLogSpace/NativeOuter.lean`, lines 50–51) yields the count word and the answer bit from the canonical search, and a transport lemma (line 73) carries both to the machine's tapes. These are theorems about the concrete step function. The arithmetic that makes them true, the sparsification, the ring and moment computations, the search, is the content of the other three thousand files, and the only claim made about it here is the one the kernel makes: Lean accepts it (Section 5).

4.6 Toolchain, Mathlib, and the build hooks

The Lean version is fixed (`lean/lean-toolchain`: `leanprover/lean4:v4.34.1`; `lean/lakefile.lean` line 8: `fixedToolchain := true`). The manifest (the list of the packages the build depends on, each at an exact version) lists 42 packages; Mathlib is pinned to upstream revision `d13f23b723b8a846827a245b89c10fc7d3f11612` at `leanprover-community/mathlib4`, and the audit of 2026-11-05 confirmed that revision on GitHub as the upstream commit that moves to `v4.34.1`. The repository carries 23 compatibility patches under `lean/patches/`, **none for Mathlib**; every one is for a third-party package that this closure does not import. The build file has two hooks, pieces of code that run automatically when the build tool configures or updates the package: one clones and patches eleven dependencies, the other verifies or applies the patches for the rest (Appendix B.4). They touch only the dependency directory, and only non-Mathlib packages. They are one reason the inspection did not build, and the reason the replay of Section 5 runs inside a sandbox with the network on.

4.7 What the repository does not record

Three things one would expect beside a verified refutation of a forty-year-old hypothesis are absent. Their absence is a fact about evidence, not about the proof.

- **No saved axiom check.** `SqrtLogSpace/Final.lean` ends with five `#print axioms` commands (lines 197–201), and seven other files of the closure carry fourteen more (nineteen commands in eight files; the family's directory holds seventeen such files, but nine of them are among the 886 never built). Their output exists only while the build runs and is recorded nowhere: a search of the whole clone for the string `depends on axioms` finds nothing.
- **No automated build.** `.github/workflows/` holds one file, `lock-prs.yml`, which locks pull-request conversations. Nothing builds or checks the Lean.
- **No second checker for this family.** The comparator's JSON (`lean/ComparatorChallenges/SubexponentialSAT.json`) names the one theorem, an empty `definition_names` list, the three permitted axioms `propext`, `Quot.sound`, `Classical.choice`, and `enable_nanoda: false`. Nanoda is an independent checker written in another language (Section 5); with it off, the collection's own check of this family rests on one implementation of the kernel. Of the collection's 418 challenge JSONs, 413 set `enable_nanoda` to `false`, two to `true`, and three omit the key. The

Table 4.2: The trust boundary before and after the replay of Section 5. "Moved" means the item no longer has to be taken on trust; "not moved" means it still does.

item	status after the replay
(i) the build	**moved**: the 3,091-module closure built from source in our sandbox at the pinned toolchain, exit 0, no `error`, no `sorry`.
(ii) the axiom output	**moved**: `#print axioms` of the comparator's theorem printed `[propext, Classical.choice, Quot.sound]`, and the nineteen in-build axiom lines say the same; the output is on record.
(iii) kernel and Mathlib	**moved in part**: Mathlib was fetched from upstream at the pinned revision and not patched, so "as upstream" is now observed rather than assumed; the Lean kernel of `v4.34.1` is still trusted as upstream, now beside a second checker (v).
(iv) the comparator run	**not moved**: the statement-level match through `lean4export` was not performed; the match this paper relies on is the textual one of Section 4.4.
(v) a second checker	**moved**: `nanoda_lib` checked the exported dependencies of the theorem, 95,176 declarations, with no errors and the three axioms; what this adds and what it does not is stated in Section 5.

comparator's README describes the check as: build challenge and solution in a sandbox, export both, match each named theorem's statement with every declaration it uses, confine the proof to the permitted axioms, and have the Lean kernel accept it; with nanoda on, the exported environment is replayed through the independent checker as well (ammkrn, 2026). Section 9 records the versions.

Two smaller consistency facts, for the record. The manuscript's source is dated "17 October 2026, revised 25 October 2026" (`preprints/The-Exponential-Time-Hypothesis-is-False-October-25-2026/build/main_full_version.tex`, line 25); the collection's history file mentions the family once, in the sentence that names the working group at whose request the materials were released (`history.md`, line 18), and its only "Formalization Fix" entry (line 7) concerns another family. And the scope note `lean/docs/378.md` states the verified exponent as $n(\log\log n)^2/\sqrt{\log n}$ (line 9), which is what the library proves and what Section 3 compares with the manuscript's.

4.8 The trust boundary, joined with the replay

Before anything was run, the proof's trust rested on five things: (i) OpenAI's unpublished build, since nothing showed the closure type-checks; (ii) the absence of any recorded axiom output; (iii) the Lean kernel of `v4.34.1` and Mathlib `d13f23b7` being what upstream publishes; (iv) the comparator run, not reproduced; (v) no second checker for this family. Table 4.2 says which of the five Section 5 moved.

After the replay, then, the proof's trust rests on four things: the Lean kernel of `v4.34.1` *and* nanoda agreeing on the exported dependencies of the theorem; the export tool `lean4export` writing out what the built environment (the set of all checked declarations) holds; upstream Mathlib at the pinned revision; and the textual, not export-level, match between the solution's statement and the challenge's. Everything the inspection could find by reading, the build and

the two checkers confirmed. Nothing the inspection could not settle by reading has been settled by anything else.

4.9 What this does not claim

- **The arithmetic was not read.** The inspection followed the spine from `SqrtLogSpace/Final.lean` down to the compile lemmas and swept the whole closure by keyword; the content of the other three thousand files was not read. The argument above is that its correctness is enforced by the kernel, not that we verified it.
- **A text search is necessary, not sufficient.** Zero `sorry` and zero `axiom` by text search cannot show that a file type-checks; only the build does (Section 5). And a keyword count is over source text at one commit; a proof can be weakened only through the mechanisms searched for, and we searched for the ones we know.
- **The textual statement match is not the comparator's.** The comparator matches exported constants; we matched source text after normalising spacing and found it identical. A difference the normalisation hides would have to be invisible to the kernel as well, but that is an argument, not a run.
- **Nothing about the constants' values.** The bounds $A \geq 258$ and $C \geq 28{,}701$ are what the proof's witnesses are built from; they are lower bounds on what this proof exhibits, not statements about any machine's running time, and a different proof of the same statement could exhibit different ones.
- **Nothing about the Lean kernel, Mathlib or the toolchain.** All three are trusted as upstream at the pinned versions; the second checker of Section 5 is evidence about the first, not a proof of either.
- **Nothing about what the theorem says.** That is Section 2; a sound proof of a weak bound is a sound proof of a weak bound.

5 Independent replay

The repository holds no saved output of the axiom check for the family, no automated build of its Lean, and `enable_nanoda: false` in the comparator's settings (Section 4). We therefore built the proof ourselves, in isolation, at the Lean version the repository pins, and recorded what the kernel reports. This section says exactly what was run, where, and what came out. The scripts are quoted verbatim in `paper3/replay/README.md`, and the logs are under `paper3/replay/logs/`.

5.1 What was run, and where

A fresh copy of the clone (commit `4eaee875`) was built inside a sandbox on the project's machine. The sandbox tool is `bwrap` (bubblewrap). Inside it, the system directories are read-only; the home directory is a private, empty one (a `tmpfs`), so nothing of the real home or of this repository is visible; processes, inter-process communication and host names are kept apart from

the host's (separate "namespaces"); the whole sandbox is confined to eight cores (`taskset -c 0-7`) at the lowest scheduling priority; and the network is on, because the repository's build file (its `lakefile`) clones its 41 dependencies at build time and the compiled Mathlib comes from a download cache. Inside the sandbox, the Lean installer `elan` was installed into the private home and fetched the pinned Lean version from the repository's own `lean-toolchain` file. Then, in the clone's `lean/` directory, three commands ran: `lake exe cache get`, which downloads the compiled Mathlib; `lake build OAI.Algorithms.SubexponentialSAT.Main`, which builds the solution's 3,091 files and not the collection's default target, which would build all 721 families; and `#print axioms` of the one theorem the comparator names, `OAI.Algorithms.SubexponentialSAT.SqrtLogSpace.exact_count_and_sat`, through `lake env lean`. Only text left the sandbox; nothing built inside it was executed outside it.

Two runs were needed. The first, started at 01:03 on 2026-11-05, failed at once at the build step because its command line carried `-j 8`, an option this version of the build tool does not have (the whole of its build log is the one line `error: unknown short option '-j'`); nothing of the family was compiled. The second, started at 01:16 with that option removed and the core cap moved to `taskset`, is the run reported below. Both scripts, both versions, and the launch command are in the README; the first run's logs are kept beside the second's.

5.2 What came out

The build finished with exit code 0 in 33 minutes of wall clock on eight cores (01:16:25 to 01:49:31). It compiled **3,091** modules with the prefix `OAI.Algorithms.SubexponentialSAT`: exactly the set the inspection of Section 4 enumerated, the same 3,091 names with nothing missing on either side (`paper3/replay/counts.sh`). The build tool's own job total, `12014`, counts the cached Mathlib and dependency modules as well, none of which was compiled. The log has no line containing `error` and none containing `sorry`. Its eleven `warning` lines are the build tool noting that the repository's compatibility patches leave eleven dependencies with "local changes", and none of the eleven is Mathlib.

Eight source files of the closure carry nineteen `#print axioms` commands (seventeen files of the family's directory do, but nine of those are never built), so the build log records nineteen axiom lines as it goes. Every one reads `[propext, Classical.choice, Quot.sound]`, the three standard axioms. Among them, `SqrtLogSpace/Final.lean`, lines 197–201, prints the axioms of `compiled_exact_count_and_sat`, `exact_count_and_sat`, `exact_count_eth`, `sat_polynomial_space` and `original_sat_main`. The separate `#print axioms` run after the build gives, for the comparator's theorem,

```
'OAI.Algorithms.SubexponentialSAT.SqrtLogSpace.exact_count_and_sat'
depends on axioms: [propext, Classical.choice, Quot.sound]
```

(`paper3/replay/logs/run2/axioms.txt`). The versions are Lean 4.34.1 (the log line reads `Lean (version 4.34.1, commit 5045d005..., Release)`), Lake `5.0.0-src+5045d00`, and Mathlib at revision `d13f23b723b8a846827a245b89c10fc7d3f11612` from the clone's own manifest, with its compiled files from the Mathlib cache (8,908 files). The replay copy is at commit `4eaee875`, the same as the pristine clone, and the comparator file and the solution's files have the same hashes in both (`paper3/replay/logs/run2/hashes.txt`; the comparator statement's

SHA-256 begins `4183b205`). The full build log is kept out of the repository by size; its SHA-256 (`d313f594...`) and its last 200 lines are committed.

So the sentence every later section relies on reads: *replayed*. The solution builds from source at the pinned Lean version with unpatched Mathlib, and the kernel reports the three standard axioms and nothing else for the theorem the comparator names. This moves the first three items of the inspection's trust boundary (Section 4): the build is no longer OpenAI's unpublished one, the axiom output is now on record, and the Mathlib revision was fetched from upstream and not patched.

5.3 The second checker

The comparator's settings for the family have `enable_nanoda: false`: OpenAI's check of this family ran the Lean kernel only. We ran a second checker, in the same sandbox, after the build (`paper3/replay/nanoda.sh`, recorded in the README before it ran). Two tools are involved. `lean4export` (tag `v4.34.0`, commit `076e8e57`, built with the pinned `v4.34.1` Lean so that it reads the compiled files the build produced) wrote out `exact_count_and_sat` and everything it depends on from the built environment, as a plain-text file: 2,355,666,449 bytes, 43,633,742 lines, format version 3.1.0, in 68 seconds. Then `nanoda_lib` (ammkrn, 2026) (version 0.4.31, commit `8de7e0c4`, 2026-10-29), a checker for Lean 4 proofs written in Rust that shares no code with the Lean kernel, read that file and re-checked every declaration in it, with `propext`, `Classical.choice` and `Quot.sound` as the permitted axioms and the `Nat` and `String` literal extensions on. Its output (`paper3/replay/logs/nanoda/nanoda_check.txt`):

```
Checked 95176 declarations with no errors
```

in 110 seconds on eight cores, exit code 0, after printing back the theorem's statement and the three axioms from its own reading of the file. The export contains exactly three `axiom` declarations (`grep -c '"axiom"'` on the file), so nothing else could have been permitted. The export file (SHA-256 `bdbc9afe...`, `paper3/replay/logs/nanoda/export_sha256.txt`) stays in the sandbox directory by size; the configuration, the build logs and the checker's output are committed.

5.4 What this does not claim

- The replay trusts the Lean kernel of the pinned version (`v4.34.1`) and upstream Mathlib at the pinned revision. It moves the proof's trust from "OpenAI's unpublished build" to "a build we ran", not from the Lean kernel to anything else. The second checker removes the Lean kernel from the list for the theorem's dependencies as exported, but adds `lean4export` (that it writes out what the environment holds) and `nanoda_lib` (that it checks what it reads) to it. Two independent checkers agreeing is evidence, not a proof that either is correct.

- It is not the comparator run. The comparator checks that the solution's exported statement matches the challenge's; we did not run that tool chain, and the statement-level match the paper relies on is the textual one of Section 4 (the 35 definitions and the theorem statement, identical after normalising spacing, two of them up to the `noncomputable` keyword).

- The counts above are counts of lines in one build's log on one machine with the named versions; a different build tool would print different job totals (the 12,014 includes the cached Mathlib modules), and a different machine a different time.
- Nothing here bears on what the theorem *says* (Section 2) or on the manuscript's unverified exponents (Section 3); a replayed proof of a weak bound is a replayed proof of a weak bound.

6 What follows, in general terms

Sections 2–5 read the verified statement, drew the line between it and the manuscript, inspected the proof and replayed it. This section asks what follows from the verified statement alone. Two things: which of the standard hardness hypotheses it refutes, and by what step; and what a lower bound that was derived from one of those hypotheses has lost. The manuscript answers both questions itself, in its §1.1.1–§1.1.2 (p. 4–5) and §1.1.3 (p. 5), and we quote it. What we add is the exact logical form of each hypothesis, the one line of the comparator statement that denies it, and the distinction, stated in general terms, between a theorem whose premise has become false and a statement that has been refuted. Nothing here names a problem the manuscript's §1.1.1 and §1.1.3 do not name, and nothing here is about any problem's complexity beyond what those two sections state.

A *hypothesis*, in this section, is a conjecture about how hard satisfiability is. The Exponential Time Hypothesis, ETH, says that 3-SAT needs time exponential in the number of variables. Its strong form, SETH, says that as the clause width k grows, no algorithm for k-SAT does much better than trying all 2^n assignments. Several variants exist, and the manuscript lists them. A *lower bound derived from a hypothesis* is a theorem of the form "if the hypothesis holds, then problem X has no fast algorithm"; hundreds of such theorems exist in the literature.

6.1 The one property every refutation uses

Every refutation below rests on the same parts of the verified theorem (comparator file lines 194–217, Section 2). For each fixed width $k \geq 2$ there is one deterministic machine which, given any formula of width at most k on n variables, halts with the number of satisfying assignments on its count tape (lines 203–204), the answer "is that number positive" on its answer tape and in its halting state (lines 205–207), within the bound $C\,(L+1)^d\,2^{A\rho(n)}$ (lines 210–211). Four features of that property do the work.

1. **It is deterministic.** The step rule is a function; there is no random choice and no guess. A deterministic algorithm is also a randomised algorithm, one that happens never to err, and also a nondeterministic algorithm, one that happens never to guess. So each refutation of a hypothesis about deterministic algorithms is a refutation of its randomised and nondeterministic forms by the same machine.
2. **The bound is subexponential in n alone.** A formula of width k on n variables has at most about n^k distinct clauses (2,604,376 at $n = 125$, $k = 3$), so the input length L is at most a polynomial in n, and the whole bound is a polynomial in n times $2^{A\rho(n)}$. Since $\rho(n)/n$ goes to zero (Appendix A, A.6), for every rate $\delta > 0$ there is a size beyond which the bound is

below $2^{\delta n}$; below that size the bound is a constant depending on k and δ. In the standard notation, the running time is $O_{k,\delta}(2^{\delta n})$ for every $\delta > 0$ (Appendix B.5). This is a fact about the limit, and the hypotheses are statements about the limit. It coexists with Section 2's observation that $\rho(n) > n$ on every $50 \leq n < e^{5504}$, which is about where the limit begins.

3. **One machine per k, for every n.** The machine and the constants are chosen after k and before n (Section 2); there is no $n \geq 20$ condition in the comparator statement, though the manuscript's Theorem 1.1 carries one (p. 2, lines 56–57). So the verified form is the strong form of the denial, one algorithm for every rate at once: a single algorithm that runs within $2^{\delta n}$ times a polynomial for every δ at once, where the denial of a hypothesis of the form "for every δ there is an algorithm" would have allowed a different algorithm for each δ.

4. **The machine model is the conservative one.** The comparator's machine is a multitape Turing machine that moves one head one cell or writes one symbol per step. Every step of such a machine is a bounded amount of work on any of the standard models of computation (random-access machines, other kinds of Turing machine) in which the hypotheses are stated, so the bound transfers to those models with at most a constant factor per step, which the polynomial absorbs. The input is the manuscript's own encoding, of length polynomial in $n + m$; converting from any other reasonable encoding is polynomial too.

6.2 The hypotheses, one by one

The manuscript's own sentence is (p. 4, lines 187–191):

> "Theorem 1.1 refutes ETH, SETH, Gap-ETH, and NSETH. Indeed, testing whether the exact count is positive or zero decides both satisfiability and unsatisfiability in deterministic $2^{o(n)}(n+m)^{O_k(1)}$ time for every fixed k; deciding satisfiability also solves the Gap-ETH promise problem. The deterministic algorithm also refutes the bounded-error randomized versions of ETH [DHM$^+$14, Section 1] and SETH [CDL$^+$16, Definition 2.1], and of Gap-ETH."

and, for counting (p. 5, lines 211–212): "Our algorithm (Theorem 1.1) solves #3-SAT deterministically in $2^{o(n)}\,\mathrm{poly}(n, m)$ time and therefore refutes #ETH." We take each hypothesis in the form the manuscript states it, with the manuscript's own source for that form, and say which part of the verified theorem denies it. Table 6.1 is the summary, and Appendix B.5 gives each hypothesis and its denial with the quantifiers written out. In every case the only thing used is the property of Section 6.1; nothing is used from the manuscript's sharper exponents, which are not verified (Section 3).

ETH. As stated (p. 4, lines 165–166): "The Exponential Time Hypothesis (ETH) asserted that 3-SAT requires $2^{\Omega(n)}$ deterministic time in the worst case, and hence so does k-SAT for every fixed $k \geq 3$ [IP01, Theorem 1]," citing Impagliazzo and Paturi (2001). In words: there is some fixed rate $\delta > 0$ such that no deterministic algorithm decides 3-SAT within $2^{\delta n}$ times a polynomial, on all inputs. Its denial: for every rate $\delta > 0$ there is such an algorithm. The verified statement at $k = 3$ supplies, for every δ, the same machine, whose answer bit is the decision, within that time (item 2 above). This holds on every input of every size, so it refutes the hypothesis whether "requires $2^{\Omega(n)}$ time" is read as a lower bound at infinitely many sizes or at all large sizes. In the vocabulary of Impagliazzo and Paturi (2001), where s_k is the smallest rate δ such that k-SAT has an algorithm within $2^{\delta n}$ and ETH is the statement that $s_3 > 0$: the verified statement gives $s_k = 0$ for every $k \geq 2$, by one algorithm per k.

SETH. As stated (p. 4, lines 166–169): "The Strong Exponential Time Hypothesis (SETH) made the sharper prediction that k-SAT requires $2^{n(1-o_k(1))}$ time as k grows. Precisely, for every $\varepsilon \in (0,1)$, there is a fixed k for which no deterministic algorithm solves k-SAT in $2^{(1-\varepsilon)n}(n+m)^{O(1)}$ time," with the motivation attributed to Impagliazzo and Paturi (2001)'s open problems (lines 169–171). In words: however small a saving ε one asks for over trying all 2^n assignments, some clause width k makes that saving impossible. Its denial: there is a saving ε that every fixed width k allows. The verified statement gives more: for *every* saving ε and every $k \geq 2$, the machine for k runs within $2^{(1-\varepsilon)n}$ times a polynomial for all large n, because $\rho(n)$ grows more slowly than n; below that size the exponential factor is a constant. The polynomial's degree depends on k, and that is exactly what the hypothesis's "for a fixed k" permits. The refutation does not pass through ETH; it reads the statement at each k. It is also standard that SETH implies ETH, through the Sparsification Lemma of Impagliazzo et al. (2001), a tool that trades one formula with many clauses for a few formulas with few clauses (Appendix B.5); so the refutation of ETH alone would already refute SETH. We note the implication for the record; the direct reading above is what the paper rests on.

Gap-ETH, and its randomised form. As stated (p. 4, lines 172–174): "The Gap Exponential Time Hypothesis (Gap-ETH) strengthens ETH by postulating constants $\delta, \gamma > 0$ for which no $O(2^{\gamma n})$-time algorithm can distinguish a satisfiable 3-CNF from one in which every assignment violates at least a δ fraction of the clauses, even when $m = O(n)$ [Din16, MR17]," citing Dinur (2016) and Manurangsi and Raghavendra (2017); and (lines 175–176) "The randomized version rules out an algorithm with the same running time that distinguishes the two cases with probability at least $2/3$ on every promised input [CCK$^+$20, Conjecture 2.5]," citing Chalermsook et al. (2020). In words: it should be hard even to tell a satisfiable formula from one that is far from satisfiable. Its denial: for every rate γ there is an algorithm within $O(2^{\gamma n})$ that tells the two apart, already when the number of clauses is proportional to n. Exact decision tells them apart, since a formula in which every assignment violates some clauses is unsatisfiable; and the verified machine at $k = 3$ decides within $O(2^{\gamma n})$ for every γ, at every number of clauses (item 2). The promise is never used: the machine answers correctly on every input, promised or not, with probability 1, which refutes the randomised form by item 1.

NSETH. As stated (p. 4, lines 181–183): "The Nondeterministic Strong Exponential Time Hypothesis (NSETH) asserts that, for every $\varepsilon > 0$, some fixed k admits no nondeterministic $2^{(1-\varepsilon)n}$ $(n+m)^{O(1)}$-time algorithm for deciding whether a k-CNF is unsatisfiable, or equivalently whether a k-DNF is a tautology [CGI$^+$16]," citing Carmosino et al. (2016). (A k-DNF is the mirror image of a k-CNF, an "or" of small "and"s, and a tautology is a formula true under every assignment.) In words: even an algorithm allowed to guess should not be able to certify unsatisfiability with a saving over 2^n. Its denial: there is a saving that every fixed k allows, for a guessing algorithm. The verified machine's answer bit is `false` exactly when the count is zero (lines 205–207), so it decides unsatisfiability deterministically within the SETH bound of the previous paragraph for every ε and every k; and a deterministic machine is a guessing machine that never guesses, so every convention for nondeterministic time is met. The manuscript's "decides both satisfiability and unsatisfiability" (line 188) is this.

#ETH. As stated (p. 4, lines 204–205): "The counting Exponential Time Hypothesis (#ETH) asserted that exactly counting the satisfying assignments of a 3-CNF requires $2^{\Omega(n)}$ deterministic time [DHM$^+$14, Section 1]," citing Dell et al. (2014). Its denial: for every rate $\delta > 0$ there is a deterministic algorithm within $O(2^{\delta n})$ that outputs the exact count. The verified machine at $k = 3$ halts with the count on its count tape, a binary number of at most $n + 1$ bits (lines 203–204, 208–209), within the same bound. This is the one hypothesis for which the yes-or-no bit is not enough and the count tape is what refutes it. It is also the hypothesis the comparator statement was written for: the count is the theorem's first conclusion and the decision its corollary.

The bounded-error randomised ETH and SETH. The manuscript names them by citation only (p. 4, lines 190–191): "[DHM$^+$14, Section 1]" for ETH (Dell et al., 2014) and "[CDL$^+$16, Definition 2.1]" for SETH (Cygan et al., 2016). In each, the algorithm may toss coins and must be right with probability clearly above one half on every input. A deterministic algorithm is such an algorithm that is right with probability 1 (item 1), so the refutations of ETH and SETH above refute these forms by the same machine, with no change to the bound. We have not re-read the two sources' definitions; we state the forms as the manuscript cites them, and a reader who needs the exact error threshold or the exact model of the source should go to the cited section and definition.

Table 6.1: The hypotheses the manuscript's §1.1.1–§1.1.2 list, each in the form the manuscript states it (page of `paper.pdf`, lines of the committed extract), with the manuscript's own source for that form, the logical form in symbols, and the part of the verified theorem that denies it (lines 194–217 of the comparator file at `4eaee875`). A, C_k, d_k are the never-computed constants of Section 2. Generated, with every page and fragment checked against the extract and the clone, by `python3 -I paper3/scripts/consequences.py`.

hypothesis	stated	source	form	negated by
ETH	p. 4, 165–166	IP01, Theorem 1	$\exists\,\delta > 0$: no deterministic $2^{\delta n}$ poly algorithm decides 3-SAT	the answer bit at $k = 3$: deterministic, $2^{A\rho(n)}$ poly, $\rho(n) = o(n)$
SETH	p. 4, 166–169	IP01, Open Problems, item 2	$\forall\,\varepsilon \in (0,1)\ \exists k$: no deterministic $2^{(1-\varepsilon)n}(n+m)^{O(1)}$ algorithm for k-SAT	the answer bit at every k: for each k, $A_k\rho(n) \le (1-\varepsilon)n$ for all large n
Gap-ETH	p. 4, 172–174	Din16, MR17	$\exists\,\delta, \gamma > 0$: no $O(2^{\gamma n})$ algorithm separates satisfiable 3-CNFs from δ-far ones, even at $m = O(n)$	exact decision separates the two cases; at $m = O(n)$ the bound is $2^{A\rho(n)}\,\mathrm{poly}(n) = O(2^{\gamma n})$ for every γ
randomised Gap-ETH	p. 4, 175–176	CCK+20, Conjecture 2.5	as Gap-ETH, for algorithms correct with probability $\ge 2/3$ on every promised input	a deterministic algorithm is correct with probability 1
NSETH	p. 4, 181–183	CGI+16	$\forall\,\varepsilon > 0\ \exists k$: no nondeterministic $2^{(1-\varepsilon)n}(n+m)^{O(1)}$ algorithm decides k-UNSAT	the answer bit is `false` iff `count` $F = 0$; a deterministic machine is a nondeterministic one with no guess
randomised ETH	p. 4, 190	DHM+14, Section 1	ETH for bounded-error randomised algorithms	a deterministic algorithm is a bounded-error randomised one with error 0
randomised SETH	p. 4, 190–191	CDL+16, Definition 2.1	SETH for bounded-error randomised algorithms	as above
#ETH	p. 4, 204–205	DHM+14, Section 1	$\exists\,\delta > 0$: no deterministic $2^{\delta n}$ algorithm counts the satisfying assignments of a 3-CNF	the count tape at $k = 3$ holds `countWord` (`count` F) within the same bound

What the manuscript places beside the list. Two sentences follow the refutation paragraph (p. 4, lines 191–194): the result "does not conflict with unconditional exponential lower bounds

for restricted proof systems such as regular resolution [BI13]," and "In 2016, Ryan Williams refuted the Merlin–Arthur version of SETH (MA-SETH) by giving a protocol that certifies k-CNF unsatisfiability in $2^{n/2}\,\mathrm{poly}(n,m)$ time [Wil16, Theorem 3.3]" (Williams, 2016). Both are context. The first says that a lower bound proved outright for one restricted way of writing proofs of unsatisfiability ("regular resolution" is one such way) is not a statement about algorithms in general and is not touched. The second says that one member of the family, the version in which the proof of unsatisfiability may be checked with the help of randomness (a Merlin–Arthur protocol), was already false before this result. Neither is used below.

6.3 What a derived lower bound loses

The manuscript's §1.1.3 opens with a list of lower bounds that were proved from ETH or from SETH (p. 5, lines 213–225), for problems with exponential algorithms and for problems with polynomial ones, and then says what its result does to them (lines 226–227):

> "Our refutation of ETH and SETH removes one basis for the hardness claims above. Some of the same lower bounds follow from weaker hypotheses that may still hold."

We say what that means in general terms, for a theorem of either of the two standard shapes,

(E) ETH $\Rightarrow$ no $2^{o(n)}$ algorithm for X,

(S) SETH $\Rightarrow$ no $(c-\varepsilon)^k$ algorithm for Y for any $\varepsilon > 0$,

where n and k are whatever measure of the input the theorem names and c is its constant. In words: (E) says that if ETH holds, problem X has no subexponential algorithm; (S) says that if SETH holds, no algorithm for problem Y beats the base c. Nothing below depends on what X and Y are.

The theorem stands; its premise is false; it derives nothing. A theorem of shape (E) or (S) is an "if … then". It is proved by a reduction: a recipe that turns a fast algorithm for X or Y into a fast algorithm for k-SAT. The recipe is unaffected by anything in the verified statement; the "if … then" remains a theorem. What has changed is that its "if" is now known to be false (Section 6.2), so the theorem can no longer be applied: it derives nothing about X or Y. That is the whole of what "removes one basis" means.

The conclusion is not refuted. A false "if" does not make the "then" false. The statement "no $2^{o(n)}$ algorithm for X" and the statement "no $(c-\varepsilon)^k$ algorithm for Y" are untouched by anything in the comparator statement, which speaks about k-CNF formulas and nothing else. To refute either, one would need an algorithm for X or Y, and the verified statement supplies one only through a reduction in the opposite direction to the one the lower bound used. The manuscript says exactly this (p. 5, lines 237–238): "To obtain a faster algorithm for a target problem from our SAT algorithm, we need a reduction in the other direction, from that problem to fixed-width SAT, that controls the number of variables." And it places the general caveat on itself at p. 2, lines 95–99:

> "Note that runtime bounds of the form $2^{o(n)}$ do not transfer under general polynomial-time reductions ($2^{n^{o(1)}}$ would be needed for this), so our result does not imply that general NP problems admit subexponential time algorithms."

So for shape (E): a subexponential algorithm for X follows only from a reduction of X to fixed-width SAT in which the number of SAT variables is proportional to X's own size measure, and the manuscript gives one such transfer (below). For shape (S): an algorithm for Y with base below c would need the SAT instance to be solved with an exponent a fixed fraction below $k \log_2 c$; the verified exponent is A times ρ of the SAT instance's variable count, A is unknown with no value below 257 on record, and nothing of the kind follows (Appendix B.5). No statement of shape "no $(c-\varepsilon)^k$ algorithm" is refuted by anything in this paper or, so far as we can read, in the manuscript.

Hardness as a classification. A statement of the form "Y is SETH-hard" (or ETH-hard) records that a reduction exists from k-SAT to Y with the stated control of the size measure. The reduction still exists, and the statement is still true as a statement about reductions. What it used to convey, that a fast algorithm for Y is unlikely because it would refute SETH, it no longer conveys: a fast algorithm for Y would refute SETH, and SETH is refuted. The classification is now a fact about reductions and no longer a fact about Y's running time.

The weaker hypotheses the manuscript names are untouched. The manuscript gives two (p. 5, lines 229–236). The first is the Orthogonal Vectors Conjecture, which it states and then relates to SETH: "The reduction from SAT to Orthogonal Vectors shows that SETH implies OVC [Wil05, VW19]" (lines 231–232). That implication is a theorem with a false premise, like (E) and (S): it says nothing about OVC either way, and nothing in the verified statement is about orthogonal vectors. The second is the form of SETH for formulas of unrestricted width, which we quote with the manuscript's list of the problems it covers elided (lines 233–236):

> "The lower bounds for [...] also follow from the hypothesis that unrestricted CNF-SAT admits no $2^{(1-\delta)n}\,\mathrm{poly}(n,m)$-time algorithm for any constant $\delta > 0$ [Bri14, RV13, LMS18]. Theorem 1.1 gives a running-time guarantee for each fixed clause width k, while this hypothesis allows the width to grow with n."

The verified statement cannot reach this hypothesis, and the reason is visible in the order of its words (Section 2): the machine and the constants C, A, S, d, e are chosen after k, and nothing in the statement bounds how they grow with k. For a formula whose width grows with n there is no single machine and no single bound in the statement at all; even the trivial observation that every k-CNF is a CNF runs the wrong way, since the hypothesis speaks about all widths at once. So the unrestricted-width hypothesis, and every lower bound that follows from it, is exactly where it was.

What a transferred algorithm inherits. The manuscript's one positive transfer is Set Splitting with sets of size at most k (given small sets over a universe of elements, colour the elements with two colours so that no set is all one colour; p. 5, lines 239–243): one variable per element and two clauses per set give a k-CNF with "one variable per universe element, so for fixed k Theorem 1.1 gives subexponential time in the universe size, up to a polynomial input-size factor," citing Cygan et al. (2016)'s Observation 3.6. Section 3 tags this supported: the encoding is on paper, the counter is the verified one. What such a transfer inherits is the verified guarantee on the SAT instance it builds, $C(L+1)^d\, 2^{A\rho(n')}$ with n' the number of SAT variables, and therefore the title's limitation with it: at $A = 1$ the exponential factor is above $2^{n'}$ for every $50 \le n' < e^{5504}$

(Table 2.1), with C, A, d unknown. In general terms: an algorithm obtained from the verified statement through a variable-controlling reduction is a subexponential algorithm whose verified guarantee is weaker, at every practical size, than trying every assignment of the SAT instance it constructs. That is not a defect of the transfer; it is the same sentence as the title, read through a reduction.

6.4 What this does not claim

- **No statement "no $(c-\varepsilon)^k$ algorithm" is refuted**, for any problem, any size measure or any constant c. Such a statement, proved from SETH, has lost its proof and kept its truth value, which is unknown.
- **No statement "no $2^{o(n)}$ algorithm for X" is refuted** either, unless X has a reduction to fixed-width SAT that controls the number of variables; the manuscript names one such problem (Set Splitting) and we add none.
- **The unrestricted-width hypothesis is untouched**, and so is the Orthogonal Vectors Conjecture, and so is every lower bound proved from either; the verified statement is for each fixed k and says nothing about widths that grow with n.
- **Nothing about any specific problem's complexity** beyond what the manuscript's §1.1.1 and §1.1.3 state and we quote. The problems the manuscript lists at lines 213–225 and 233–235 are not discussed here, and no example from any other setting is given, by the scope of this paper.
- **No re-reading of the sources.** Each hypothesis is stated in the form the manuscript gives it, with the manuscript's citation; we did not check that the cited sections and definitions state it in the same words. If a source's form is stronger than the manuscript's, the refutation of the manuscript's form may not reach it, and the reader should check the source.
- **About the limit only.** Every refutation here denies a statement about the limit and uses only that $\rho(n)$ grows more slowly than n. None says that any machine decides or counts anything fast at any size; the constants are as unknown here as in Section 2, and the verified bound is above 2^n on every $50 \le n < e^{5504}$.
- **Nothing beyond the verified statement.** The manuscript's sharper exponents (Section 3) are not used; where the manuscript derives a consequence from them, it is not a consequence of what is verified and is not claimed here.

7 For the practitioner

The title compares two guarantees. One is the verified bound, $C(L+1)^d\, 2^{A\rho(n)}$, with C, A and d never computed (comparator file lines 136–139 and 210–211, Section 2). The other is the trivial bound for trying every assignment, 2^n times a polynomial. Neither is what a practitioner runs. What runs on a real machine, when the question is whether a formula can be satisfied, is a *SAT solver*; the standard kind is called CDCL, for conflict-driven clause learning, a search that learns from each dead end it reaches. When the question is how many satisfying assignments there are, what runs is a *model counter*, exact or approximate. No implementation of the verified machine

exists to run beside them. This section measures the tools a practitioner has, on random formulas, and draws the two guarantees next to the measurements as shapes, so that the reader can see where each sits. The point is small and the section says it once: the verified guarantee is not the binding one on any machine, because it is above 2^n for every $n \geq 50$ (Section 2), while the solvers are many orders of magnitude below 2^n at every size they were run at.

7.1 Why the comparison is between guarantees and measurements

The verified statement says a machine *exists* (comparator line 195 opens with the existence quantifier, Section 2). The solution's 3,977 files contain no command to run anything: no `#eval`, no `def main` and no input-output (`IO`) action. The script of this section checks that, reading every file of the family with `open()` and counting those three patterns: 0, 0, 0. The library proves that a machine with the stated bound exists and exhibits one as a mathematical term, inside a development with 1,048 `noncomputable` declarations and no way to execute anything (Section 4); nothing in the clone runs it on an input, and we found no implementation elsewhere. So the verified algorithm's actual running time on any formula is unknown, in both directions: nothing says it is slow, and nothing says it is fast. What is known is its *guarantee*, and a guarantee is compared with other guarantees. The measurements below are of other programs entirely. They are in the section because the question a practitioner asks is not "which guarantee is smaller" but "what would I run", and the answer to that has not changed.

7.2 What was run

Instances. Random 3-CNF on n variables with $m = \lfloor 4.26\,n \rceil$ clauses, each clause three distinct variables with random signs, generated by the script from (n, seed); twenty seeds per size; $n = 20, 30, 40, 50, 60, 80, 100, 150, 200, 250, 300, 400, 500$. The ratio 4.26 clauses per variable is where random 3-SAT switches from mostly satisfiable to mostly unsatisfiable (Selman et al., 1996); random formulas are hardest to decide there (a method from statistical physics puts the threshold at 4.2667, Mertens et al., 2006). The sizes are chosen so that the smallest ones are where the title's crossing is ($n = 50$) and the largest are where a CDCL solver on random formulas begins to take seconds. These are random instances only; nothing is claimed about formulas with structure.

Tools. CaDiCaL 1.9.5 (Biere et al., 2024), a CDCL solver, through python-sat 1.9.dev15 (Ignatiev et al., 2018), one `solve()` call, for the yes-or-no answer; Ganak 2.8.0 (Sharma et al., 2019), an exact model counter, through the pyganak binding (which runs a simplification pass called Arjun and then the counter), for the exact count; ApproxMC 4.4.0 (Chakraborty et al., 2013; Soos and Meel, 2019), an approximate counter, at its defaults $\varepsilon = 0.8$, $\delta = 0.2$ (an answer within a factor 1.8 of the truth with probability 0.8), with the count taken over all variables (its "projection set"); and, at $n \leq 30$ only, exact counting by listing every solution with CaDiCaL, one at a time, which is not a counter at any larger size (its time grows with the count, and the count grows exponentially in n) and is here as the independent check of the two counters where it can reach. Ganak and ApproxMC were installed from PyPI for this section; no `sharpSAT`, `d4` or other counter binary is on the host.

Protocol. Each call runs in its own interpreter under `nice -n 19`, at most eight at a time on the thirty-two cores of an AMD Ryzen 9 9950X (Linux 7.0.0, Python 3.12.13; the machine was shared

with other work, load average at the start in `tables/practitioner_meta.json`), with a wall-clock budget of 60 s per call. The time recorded is the wall clock (`perf_counter`) around the construction of the solver object and the one call, inside the child process, with the interpreter's start-up and the import outside it. A call over budget is killed and recorded as *censored*, that is, as "more than 60 s". A size at which a tool finishes fewer than ten of its twenty instances ends that tool's climb to larger sizes. The whole run took about twenty minutes (1,172 s).

7.3 What came out

Table 7.1 gives, per size, the number of satisfiable instances, CaDiCaL's median and maximum over the twenty seeds, the number of instances each counter finished with its median and maximum (a median over all twenty with censored runs counted as above the budget, so a median is a number whenever more than ten finished), and the two guarantees in the common scale of the next subsection. Figure 7.1 draws the same.

Table 7.1: Random 3-CNF at $m = \lfloor 4.26\,n \rfloor$, twenty seeds per size, one machine, 60 s per call. Columns: n variables; m clauses; "sat", the number of satisfiable instances among those CaDiCaL finished; for each tool, "fin", the number of instances it finished within budget, and "med" and "max", its median and maximum seconds over all twenty seeds with a censored run counted as > 60. The last two columns are the two guarantees as shapes: 2^n (trying every assignment) and $2^{\rho(n)}$ (the verified bound), both with constant 1 and both multiplied by one common factor $t_{20}/2^{20}$ that puts 2^n through CaDiCaL's median at $n = 20$ ($t_{20} = 1.28 \times 10^{-4}$ s). They are shapes, not running times. Regenerate: `python3 -I paper3/scripts/practitioner.py`.

			CaDiCaL		Ganak			ApproxMC			guarantee, s	
n	m	sat	med	max	fin	med	max	fin	med	max	2^n	$2^{\rho(n)}$
20	85	12/20	1.3×10^{-4}	1.6×10^{-4}	20	0.0015	0.0047	20	0.0016	0.0073	1.3×10^{-4}	1.9×10^{-5}
30	128	12/20	1.6×10^{-4}	1.9×10^{-4}	20	0.0014	0.0295	20	0.0031	0.0074	0.1310	0.0254
40	170	11/20	2.0×10^{-4}	2.6×10^{-4}	20	0.0018	0.0466	20	0.0058	0.0117	134	53.18
50	213	16/20	2.2×10^{-4}	3.6×10^{-4}	20	0.0041	9.48	20	0.0044	0.0148	$10^{5.1}$	$10^{5.2}$
60	256	12/20	3.5×10^{-4}	6.9×10^{-4}	20	0.0288	9.60	20	0.0099	0.0151	$10^{8.1}$	$10^{8.7}$
80	341	12/20	6.4×10^{-4}	0.0016	20	0.0518	11.76	20	0.0133	0.0245	$10^{14.2}$	$10^{16.1}$
100	426	9/20	0.0025	0.0038	20	8.34	10.97	20	0.0219	0.1987	$10^{20.2}$	$10^{23.6}$
150	639	10/20	0.0169	0.0443	20	6.16	16.11	20	0.0595	1.01	$10^{35.2}$	$10^{43.3}$
200	852	15/20	0.0578	0.3906	20	2.37	29.05	20	0.5104	25.21	$10^{50.3}$	$10^{63.5}$
250	1065	6/20	1.47	3.57	20	17.62	45.72	18	3.64	> 60	$10^{65.3}$	$10^{84.3}$
300	1278	12/20	3.72	20.51	10	> 60	> 60	11	58.91	> 60	$10^{80.4}$	$10^{105.4}$
400	1704	10/20	> 60	> 60	0	> 60	> 60	0	> 60	> 60	$10^{110.5}$	$10^{148.4}$
500	2130	8/20	> 60	> 60	—	—	—	—	—	—	$10^{140.6}$	$10^{192.2}$

The checks. On every instance where both finished, Ganak's count equals the count by listing (40 instances at $n \leq 30$) and CaDiCaL's answer equals "count > 0"; ApproxMC's estimate is within its factor $1 + \varepsilon = 1.8$ of Ganak's on 178 of the 206 instances where both finished (it promises that with probability $1 - \delta = 0.8$ per instance). The script exits non-zero if any of these fails.

What the solvers did. CaDiCaL's median is under a millisecond up to $n = 80$, 2.5 ms at 100, 58 ms at 200, 1.5 s at 250 and 3.7 s at 300 (maximum 20.5 s); at 400 it finishes 10 of 20 within

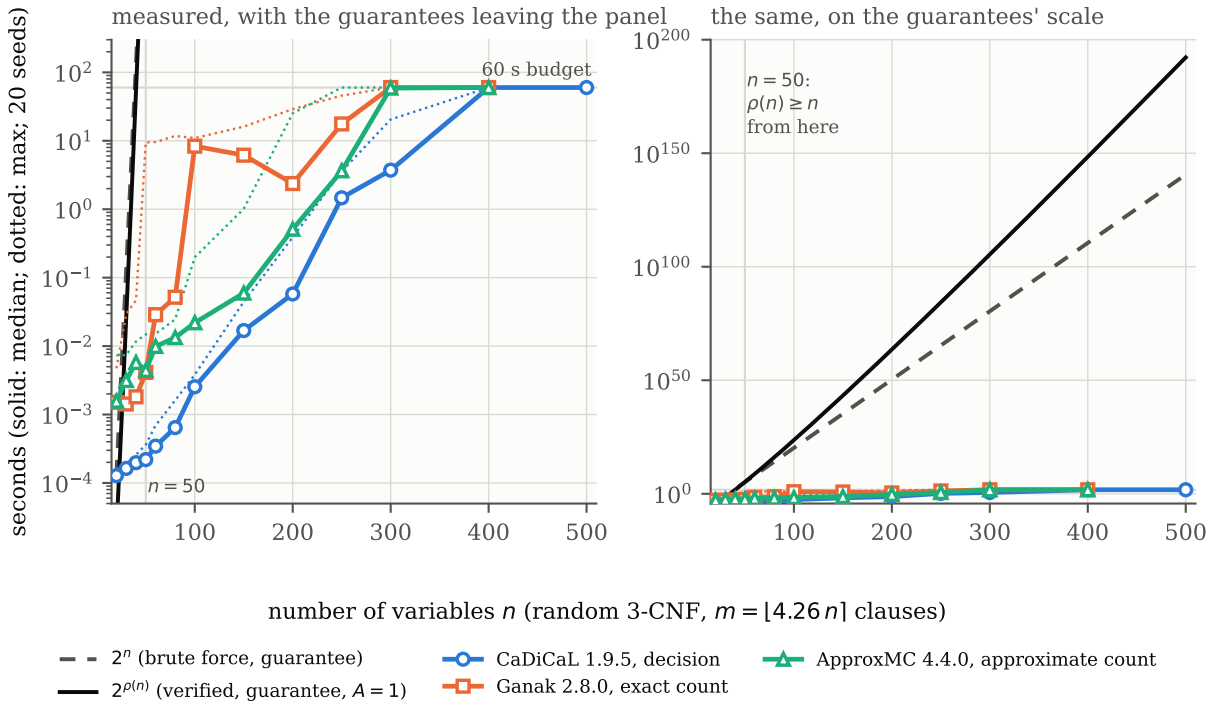


Figure 7.1: The same data on a logarithmic time axis, where each grid line is ten times the one below. Solid lines with markers are medians, dotted lines maxima, over the twenty seeds; a tool whose median is censored stops at the budget line. The two guarantees are drawn as shapes with the common factor of Table 7.1, so that 2^n passes through the $n = 20$ measurement and $2^{\rho(n)}$ crosses it at $n = 50$, where the verified exponent overtakes n (Section 2). The left panel keeps the measured seconds in view and lets the guarantees leave the frame; the right panel is the same picture on the guarantees' own scale. From `paper3/scripts/practitioner.py`.

the budget and at 500 8 of 20, every finished one satisfiable, which is the known shape of CDCL on random formulas at the threshold (the unsatisfiable half is the expensive half). Ganak finishes all 20 up to $n = 250$ (median 17.6 s there, maximum 45.7 s), 10 of 20 at 300 and none at 400; its median is flat at 8.3 s and 6.2 s at $n = 100$ and 150 and lower, 2.4 s, at 200, while its maximum rises throughout. The flat stretch looks like a fixed time budget inside the binding's preprocessing rather than counting difficulty; we report it as measured and tuned nothing (`ETH_AMBIGUOUS.MD`, 06d). Where the count is positive it is small for the size: at $n = 200$ the fifteen satisfiable instances have between 372 and 2.5×10^{12} solutions out of 2^{200} assignments. ApproxMC finishes all 20 up to $n = 200$ (median 0.51 s there), 18 at 250, 11 at 300 and none at 400. The sizes at which the three tools leave the budget, 300 for the counters and 400 for the decision, are where this measurement ends; nothing is claimed beyond them.

7.4 The two guarantees beside them

The guarantees are drawn as shapes. Both are taken with constant 1 (the polynomial factors, C, A and d are all set aside), and both are multiplied by the same factor, chosen so that 2^n passes through CaDiCaL's median at $n = 20$. Because the factor is one number for both curves, their ratio is the ratio of the guarantees themselves, $2^{\rho(n)-n}$: below 1 for $n < 50$, equal to 1 at $n = 50$ up to rounding, and above 1 from there (it is 2.8×10^3 at $n = 100$ and 4.0×10^{51} at $n = 500$). Scaling each curve to the $n = 20$ point on its own would instead make the two touch at $n = 20$ and put the verified one above from there on; the common factor is the honest choice because it keeps the crossing where the statement puts it (Appendix B.6; `ETH_AMBIGUOUS.MD`, 06a).

In that scale the brute-force guarantee reaches one second near $n = 33$ and the verified guarantee near $n = 35$. At $n = 60$, the measured CaDiCaL median is 3×10^{-4} s, against 1.4×10^8 s for 2^n and 5.4×10^8 s for $2^{\rho(n)}$: the measurement is below the two guarantees by factors of $10^{11.6}$ and $10^{12.2}$. At $n = 500$ the decision median is above the budget (8 of 20 finished within 60 s), against $10^{140.6}$ s and $10^{192.2}$ s. Nothing in those enormous factors is about the verified machine: the gap is between what a CDCL solver does on random formulas and what any exponential guarantee says, and it would be the same gap for the brute-force guarantee alone. The one fact this section adds to the title is the order of the three objects: at every size in the table the measurement is far below 2^n, and $2^{\rho(n)}$ is above 2^n from $n = 50$ on, so the verified guarantee is the largest of the three at every practical size and the measurement the smallest.

One remark on what the verified machine's *running time* would have to be, were it ever run. Nothing in this section bears on it: the constants C, A, d are unknown (Section 2), the machine is a mathematical term, and a statement that something exists is not a program. The only values on record are the proof's own, $A \geq 257$ and $C \geq 28{,}701$ (Sections 2 and 4); a guarantee of $2^{257\,\rho(n)}$ steps is a guarantee and not a measurement, and a different proof of the same statement could carry any positive constants.

7.5 What this does not claim

- **Timings on one machine, one day, named versions.** The seconds are from one AMD Ryzen 9 9950X shared with other work, under `nice -n 19` with eight calls in flight, through Python bindings; they are reproducible in shape and not in value, and the versions are those named in Section 7.2 and in `tables/practitioner_meta.json`.

- **Random instances only.** Random 3-CNF at the threshold ratio is one distribution; formulas with structure can be harder for CDCL at far smaller n and easier for counters. Nothing is claimed about any other distribution, and nothing about worst-case behaviour.
- **Nothing about the verified machine's running time.** It is unknown, in both directions; no implementation exists to run; the guarantees drawn are at constant 1 and the constants are never computed.
- **Nothing about $k > 3$.** The statement is for every fixed k; the measurement is at $k = 3$.
- **The counters are reported, not tuned.** Ganak ran through its binding at defaults, preprocessing included; ApproxMC at its default tolerance; a different configuration gives different seconds. The check against listing is at $n \leq 30$ only.
- **No statement about which tool a practitioner should use.** The section shows that the verified guarantee is not the binding one; it does not rank solvers.

8 The statement in our Lean

Sections 2 and 4 read OpenAI's statement in OpenAI's files. This section describes the one thing we wrote in Lean ourselves: a copy of the statement inside our own library, checked by the same kernel, under our own name. The point is modest. Every sentence this paper says about "the verified statement" is pinned to a named object that Lean has accepted, and anyone who wants to use the statement as an assumption later can cite that one name. Nothing is proved about the hypothesis here, and nothing is claimed beyond what the copy and its audit show.

Four words first, each in one sentence. In Lean a *Prop* is a sentence that may be true or false. A *def* gives a sentence a name without asserting it. A *theorem* asserts a sentence and carries its proof. An *axiom* asserts a sentence without a proof. Our restatement is a def: it names the sentence and asserts nothing.

The files are `lean/MOSPFormalization/SubexpPathwidth/PrimitiveMachine.lean` and `lean/MOSPFormalization/SubexpPathwidth/Frame.lean`, written on 2026-11-04 for an earlier study of ours on a different question. The directory carries that study's name. The owner chose to leave the files where they are rather than move them (`paper3/ETH_AMBIGUOUS.MD`, S3), and this paper cites nothing from that study: only the copied model, the restatement, and six small lemmas about them, all listed in Appendix A.

8.1 The copy: definitions only

The comparator file of family 378, `lean/ComparatorChallenges/SubexponentialSAT.lean`, has 220 lines. Lines 1–190 define the objects the theorem speaks about: the formulas, their encoding as a string of bits, the multitape machine, its step and its starting configuration, the exponent `runtimeScale` and the bound built from it, the count word, and the bookkeeping that records which tape cells a run has visited. Lines 191–219 state the theorem `exact_count_and_sat` and end it with `sorry`, the placeholder that a solution must replace. Line 220 closes the last namespace (a grouping of names).

Our `PrimitiveMachine.lean` is lines 1–190 and 220, and nothing else. After a thirty-line header that records the source, the commit, the file's hash and the Apache licence, the file is the

copy (Appendix A, A.1). The header says so, and a script checks it. `paper3/scripts/lean_rediff.py` reads both files with `open()` only, takes everything in ours from the line `import Mathlib` on, and compares it byte for byte with the clone's lines 1–190 followed by line 220. The diff is empty: 191 lines on each side, identical. The script also checks the clone file's SHA-256 against the hash the header records, and that the clone is at commit `4eaee875`. The log is `paper3/lean/rediff.log`.

So the copy holds the theorem's *vocabulary* and not the theorem. The `sorry` at line 218 is outside the copied range, and our file has no `sorry` and no axiom declaration anywhere (the same script counts them: zero and zero in each of our two files). Everything in the copy is a definition. A definition can be badly chosen, but it cannot be false; what the definitions mean is Section 2's subject, and the copy adds nothing to it.

8.2 The restatement

`Frame.lean` defines a sentence named `SubexpCount`, with the width k as its argument:

$$\texttt{SubexpCount}\ (k : \mathbb{N}) : \text{Prop}.$$

Its body is the body of the comparator theorem for width k. The theorem's two leading parts are handled thus: k is the argument of the def, and the condition $2 \leq k$ is carried by a second def, `ExactCountAndSat`, which says "for every $k \geq 2$, `SubexpCount` k" (Appendix A, A.2 and A.3). In words, `SubexpCount` k is the sentence of Section 2: one fixed machine, on every formula of width at most k, writes the exact number of satisfying assignments, answers whether there is one, runs within the bound built from `runtimeScale`, and visits only polynomially many cells.

The same script checks that the body is the comparator's. It takes the 23 lines after `def SubexpCount (k : ℕ) : Prop :=` in our file (lines 85–107) and the 23 lines after the theorem's leading parts in the clone (lines 195–217, with the closing `:= by` removed), and compares them in two ways: with every run of white space collapsed to one space, and line by line with the indentation stripped. Both comparisons find the two texts identical, 1,073 characters. The only difference is that each of our lines starts two spaces further left, because a def's body sits two columns inside a theorem's. We call this "character for character"; it is a textual match, by the same method as Section 4's, and it rests on the file we copied from being the comparator file at the commit we name.

Neither def is proved here, and neither is an axiom. The truth of `SubexpCount` k for $k \geq 2$ rests on OpenAI's proof and on Sections 4 and 5, not on anything in our library. Our library would be unchanged if the proof were wrong.

8.3 What is proved, and what it means in words

Six small lemmas sit beside the defs, each proved in a few lines (Appendix A, A.4–A.8). None of them says anything about any machine's running time or about whether the hypothesis is true.

- **A machine for width k is a machine for every smaller width.** `SubexpCount.anti`: if `SubexpCount` k holds and $k' \leq k$ then `SubexpCount` k' holds, with the same machine and the same constants, because a formula whose clauses have at most k' members has at most k. `ExactCountAndSat.subexpCount` unpacks the "for every $k \geq 2$" form.

- **The bound is what it is.** `runtimeBound_eq`: the wrapper `runtimeBound` $C\,A\,d\,L\,n$ unfolds, by definition, to $C\,(L+1)^d\,2^{A\cdot\rho(n)}$, the formula Section 2 reads. The proof is `rfl`: the two sides are the same term.

- **The exponent grows more slowly than n.** `tendsto_log_sq_div_sqrt` says $(\ln y)^2/\sqrt{y} \to 0$ as $y \to \infty$, and `runtimeScale_isLittleO` says that $\rho(n)/n \to 0$ as $n \to \infty$, which is written $\rho(n) = o(n)$ (Appendix A, A.6). This is the limit fact that Sections 2, 3 and 6 cite: in the limit the exponent is smaller than n by any factor one names. It is true at the same time as the title's fact that $\rho(n) > n$ for every $50 \le n < e^{5504}$, and the Lean says nothing about where the limit begins to bite. `isSubexpExponent_runtimeScale` is the same statement for $A \cdot \rho(n)$, for every constant A (Appendix A, A.7).

- **The time bound is a bound on the plain machine.** `Trace.config_of_locatedStep`: the comparator states its time bound on the *located* step, which is the plain step plus the bookkeeping of visited cells beside each configuration. A run of the located step projects to a run of the plain step with the same number of steps, so the bound is a bound on the machine itself (Appendix A, A.8). The one-line `LocatedCfg.monitor_config` is its helper.

8.4 The axiom audit

Lean records, for every proved statement, the axioms its proof uses. Three are standard in Mathlib and in every proof of Sections 4 and 5: `propext`, `Classical.choice` and `Quot.sound`. A fourth, `sorryAx`, appears exactly when a proof was left out with `sorry`. The audit `paper3/lean/axiom_check.lean` prints the axioms of 21 declarations: the ten copied definitions the statement rests on, the two defs, and the nine lemmas and helper definitions, in the pattern of our first paper's audit. It is run from our library's directory as

```
cd lean && lake env lean ../paper3/lean/axiom_check.lean \
  > ../paper3/lean/axiom_check.log
```

and the log is committed. The result: nineteen lines on exactly the three standard axioms; `Trace` on no axiom at all (it is defined by construction rules and carries no proof); `countWord` on two of the three (a proof may use fewer than the three, and fewer is never a concern); `sorryAx` on none. The library it runs against is built with Lean 4.34.0-rc2 and Mathlib at `1192d624`, neither of which is what the clone pins (Lean 4.34.1, Mathlib `d13f23b7`, Section 4). That the copied model is accepted unchanged by a different Lean version and a different Mathlib revision is small evidence that it rests on nothing patched or private, and no more than that. Our whole library builds with no `sorry` and no axiom declaration, which the loop's gate checks after every session.

8.5 What this does not claim

- **`SubexpCount` k is a def of a Prop, never proved here.** Its truth is OpenAI's theorem, replayed in Section 5; our library neither proves it nor assumes it. Nothing in this section makes the refutation more or less certain.

- **The copy is definitions only.** The proof is not copied, and could not be: it is 3,091 modules (Section 4). What is copied is the vocabulary of the statement, so that the restatement reads as the comparator does.
- **No Lean computation evaluates the exponent.** `runtimeScale` is defined over the real numbers with real logarithms and is marked `noncomputable`; no `#eval` can print $\rho(50)$ or $\rho(50)/50$. Every number in Section 2 comes from a Python script, not from Lean, and the Lean proves only the limit.
- **The match is textual.** "Character for character" means the normalised texts are equal; it is not a check at the level of `lean4export`, and it rests on the clone being the repository at the commit named.
- **The lemmas are trivial on purpose.** They fix the shape of the bound and the limit of the exponent, and nothing else. No theorem about any machine's running time, about the constants, or about the manuscript's sharper exponents is stated, because none is verified.
- **The directory name is not a citation.** The files sit in a directory named for another study of ours; nothing from that study enters this paper, and the owner's decision to leave them in place is recorded, not argued.

9 Related work and context

This section places the paper beside what others have written about the same family and about the mechanism that checked it. There are five things to place it beside: what the collection says about itself; the two papers that stated the hypotheses; a published human audit of an earlier batch of OpenAI's results; the checking tool, called the comparator; and the second proof checker, called nanoda, that we ran in Section 5. The section ends by saying where we looked for an earlier statement of the title's observation and what we found. Table 9.1 lists every source with where it is on disk, when it was fetched, and its hash or commit. We quote only what is on disk; nothing in this section is quoted from memory or from the press.

9.1 What the collection says about itself

The collection keeps a dated log, `history.md` (56 lines). Under the heading "October 26, 2026" (line 11), the entry for this family is one bullet, which we quote in full (line 18):

> "*The Exponential Time Hypothesis is False.* The paper gives a deterministic algorithm that exactly counts satisfying assignments of fixed-width CNF formulas in subexponential time and polynomial space. The case of 3-SAT refutes the Exponential Time Hypothesis. An independent working group of computer scientists led by Russell Impagliazzo, Ramamohan Paturi, and Ryan Williams will examine and explain the proof. At the group's request, we are making the materials public immediately so others can examine them as well."

Three things in this sentence matter for us. First, the group it names is led by two of the three authors of the papers that stated the hypotheses (Section 9.2). Second, the verb is "will": the log announces an examination to come. Nothing from that group is in the clone, and we fetched nothing from it, so this paper has no finding of theirs to cite, agree with or disagree with. Whatever

Table 9.1: The sources of this section. "Clone" is the read-only copy of the OpenAI collection at commit `4eaee875`; "fetched" files were downloaded on 2026-11-05 and are kept under `paper3/related/fetched/` with their SHA-256 hashes. Written by `paper3/scripts/related_work.py`.

Source	Where it is on disk	When, and at what commit	SHA-256 (first 12)
OpenAI collection, `history.md` line 18	clone, commit `4eaee875`	2026-10-29 (clone)	–
OpenAI collection, `lean/docs/378.md`	clone, commit `4eaee875`	2026-10-29 (clone)	–
OpenAI collection, `lean/ComparatorChallenges/README.md` and `SubexponentialSAT.json`	clone, commit `4eaee875`	2026-10-29 (clone)	–
arXiv 2608.14673, API record (v3)	`related/fetched/arxiv_2608.14673_api.xml`	fetched 2026-11-05	2737bacdb870
arXiv 2608.14673, abstract page	`related/fetched/arxiv_2608.14673_abs.html`	fetched 2026-11-05	9047b0010cc3
`leanprover/comparator` README (`master`)	`related/fetched/comparator_README_master.md`	fetched 2026-11-05; commit `865edc14` of 2026-10-14; archived	39e03761a9b2
`ammkrn/nanoda_lib` README (`master`)	`related/fetched/nanoda_README_master.md`	fetched 2026-11-05; commit `8de7e0c4` of 2026-10-29 (the replay's)	d4d3f3be41ba
Impagliazzo & Paturi 2001; Impagliazzo, Paturi & Zane 2001	not held; manuscript extract lines 3733–3736	–	–

that group publishes about the proof will carry more weight on the proof than Section 4 does, which only inspected and replayed. Third, the log says "the paper gives ... subexponential time" with no exponent; the exponent is in the scope note.

That scope note is `lean/docs/378.md`, seventeen lines, quoted in full in Section 3. It is the collection's own statement of the gap between what Lean proves (the exponent with the squared log log factor) and what the manuscript claims (two sharper exponents). Its table (line 17) links the result to the one comparator statement, `lean/ComparatorChallenges/SubexponentialSAT.lean`. So the collection itself records that the verified exponent is the weaker one. What it does not record is what the weaker exponent means at any size: that $\rho(n)$, the verified exponent, is larger than n for every n between 50 and e^{5504}, so that the verified bound with constant one sits above 2^n at every size anyone will ever run (Section 2). That is the title's observation, and the note does not make it.

The third document is `lean/ComparatorChallenges/README.md`, ten lines. It says how the collection's checks are run: "Install `comparator`, `landrun`, and `lean4export`, and make them available on `PATH`." (line 3), then from the `lean/` directory `lake update`, `lake exe cache get`, and `lake env comparator` on a family's JSON (lines 7–9; the example is another family's). The JSON for this family has six fields (Section 9.4 reads them).

9.2 The two papers that stated the hypotheses

ETH, the Exponential Time Hypothesis, says that 3-SAT (deciding whether a formula whose clauses have three literals each can be satisfied) has no algorithm whose running time grows slower than every exponential in the number of variables. SETH, the Strong form, says that as the clause width grows, the best possible base of the exponential tends to 2, the base of trying every assignment. Both are from Impagliazzo and Paturi (2001); the standard setting of SETH, and the lemma that lets one move between the number of variables and the number of clauses, are from Impagliazzo et al. (2001). The manuscript cites both as `[IP01]` and `[IPZ01]` (extract page 66, lines 3733–3736) and states each hypothesis in its own words in its §1.1.1, which Section 6 quotes with page and line. We do not hold either paper: `literature/` has no file for them, and `literature/MISSING.md` now records both with their DOIs. Every statement of a hypothesis in this paper is therefore the manuscript's statement of it, not the original's, and we quote nothing from the two originals. The manuscript's own account of what motivated SETH is one sentence we can locate (page 4, lines 170–171): the improvements of the known algorithms "over exhaustive search diminish as k grows [IP01, Open Problems, item 2]". Exhaustive search is the "trying all 2^n assignments" of the title.

9.3 A human audit of an earlier batch of results

Sienicki and Sienicki (2026) is a published human audit of OpenAI's AI-generated proofs. We fetched its arXiv abstract page and API record (v1 of 3 August 2026, v2 of 25 August, v3 of 9 September 2026; 13 pages; primary class cs.AI). It is not about this family. Its abstract says what it covers: "We assess 18 chapter-specific reviews of the ten mathematical results announced by OpenAI on 1 August 2026, alongside review standards, Lean formalizations, subsequent research, and mathematical references." The family this paper reads is dated 17 October 2026, revised 25 October, and entered the collection on 26 October (Section 9.1); it is not one of the ten, and the

abstract mentions neither it nor ETH (the script checks). So we cite the audit for its method of review and for nothing about the proof.

Four sentences of the abstract bear on how this paper was written. On restraint: "The article audits this review record without claiming a complete reconstruction of all ten proofs." On the finding: "No confirmed substantive mathematical error in a principal result remains in the examined assessments, although review depth varies and some dependencies remain partly checked." On a hazard of reading proofs through text extraction: "In Chapter 6, an apparent polarity error was withdrawn after an overbar lost during PDF extraction was recovered from the typeset source." And on what confidence should rest on: "We argue that confidence should combine formal checking, human reconstruction, independent mathematical use, and a public record supporting correction of both proofs and reviews."

The third of these is a warning we took. This paper reads the manuscript through a `pdftotext` extract, the same kind of text in which that overbar was lost. That is why every quotation from the manuscript carries the page of the PDF beside the line of the extract, why the Lean is read from its source files and never from a rendering, and why no claim in this paper rests on a formula read from the extract alone: the formulas that matter are the Lean's, re-typed in Section 2 from the `.lean` file and re-checked by the kernel in Section 8. Against the fourth sentence, this paper is one of the four things it lists, formal checking, replayed by us and by a second kernel, together with a reading of what the checked statement says. It is not a human reconstruction of the proof: Section 4 inspected the proof's files and did not re-derive the argument. We fetched the abstract only, and quote nothing from the paper's body.

9.4 The comparator

The comparator is the program the collection used to check that the solution file proves the challenge statement (Lean FRO, 2026). We fetched its README from the `master` branch of `leanprover/comparator` on 2026-11-05 (latest commit `865edc14` of 2026-10-14, "doc: migration notice"; the repository is archived and under the Apache-2.0 licence). Its first lines say the tool has moved: "This project has been merged into core as of Lean release v4.35.0 in the form of `lake comparator` and the accompanying `lake check`" (line 3). The collection pins Lean v4.34.1 (Section 4), one release earlier, so the stand-alone tool is the one its README names. The README opens: "Comparator is a trustworthy judge for Lean proofs." (line 8).

What it checks is stated as three guarantees for the theorems a JSON configuration names (lines 52–55):

> "All theorems in `Solution` that are listed in `theorem_names` are guaranteed to:
> 1. Prove the same statement as provided in `Challenge`
> 2. Use no more axioms than listed in `permitted_axioms`
> 3. Be accepted by the Lean kernel"

In plain words: the solution proves exactly the sentence the challenge posed, uses only the agreed axioms (the three standard ones in the example configuration, line 27), and passes the kernel, the small core of Lean that checks proofs. How it checks them is in its "Internals" (lines 184–200), six steps: build the challenge inside a sandbox; export its declarations with `lean4export`; do the same for the solution; "Verify that all declarations used in the statement of all relevant theorems in `Challenge` are the same as in the `Solution` environment" (lines 191–192); "Verify that the body of all relevant theorems in the `Solution` environment only uses axioms listed in

permitted_axioms" (lines 195–196); and "Replay the Solution environment into the Lean kernel" (line 197). The first guarantee is therefore a match of the statement's declarations as exported, not a textual comparison of the two files. That is the check Section 4 says we did not reproduce: our statement match is textual (Table 4.2, row (iv)).

The guarantees hold under six stated assumptions (lines 34–45). The one that concerns the kernel is the fifth: "The Lean kernel is correct (with external_kernels this can be reduced to 'At least one of the Lean kernel or the external_kernels is correct')" (lines 43–44). The sandbox is landrun (line 10), a tool that confines the build so that a malicious solution file cannot tamper with the checker; the README notes that its trusted base "naturally includes the operating system and hardware it is running on, plus its sandboxing mechanism" (line 58). External kernels are optional: "Comparator can additionally check solutions with external kernels" (line 70), and "For backwards compatibility reasons users may instead set enable_nanoda: true to obtain a config that calls nanoda_bin" (lines 86–87). The tool "was originally developed by Lean FRO, with feedback from the AIMO team, in support of the AIMO series of competitions" (lines 207–208).

For this family the configuration has six fields (lean/ComparatorChallenges/SubexponentialSAT.json): the challenge module; the solution module, OAI.Algorithms.SubexponentialSAT.Main; one theorem name, SqrtLogSpace.exact_count_and_sat; an empty definition_names; the three standard axioms as permitted_axioms; and enable_nanoda: false. There is no external_kernels field. So the check the collection configured for this family is the Lean kernel alone, under the fifth assumption in its unweakened form. Section 5 is the weakened form, supplied by us: the same theorem's dependencies through a second kernel. Our bwrap sandbox there played the role landrun plays here, for the same reason, and we did not run the comparator itself.

9.5 nanoda

nanoda is the second checker (ammkrn, 2026). Its README, fetched from the master branch of ammkrn/nanoda_lib on 2026-11-05, says what it is in one sentence (line 3): "This is an external type checker for the Lean 4 programming language and theorem prover." The repository's description line is "Library implementing type inference/checking functionality based on the Lean theorem prover"; the licence is Apache-2.0. It reads the file lean4export writes: "Export files can be created with lean4export" (line 13), and "Given that lean4export already uses json as its export format, adding a json parser as a dependency doesn't increase the overall complexity of the project" (line 23). Its axiom rule is the comparator's: "The average Lean user will want to permit the three semi-official axioms, which are Quot.sound, Classical.choice, and propext, and most export files will also rely on Lean.trustCompiler" (line 33); the export we checked contains the first three and no fourth (Section 5). Two kernel extensions for arithmetic on literals are off by default and "enable or disable the Nat and String kernel extensions" (line 39); the replay turned both on.

The latest commit of the repository at the time of the fetch is 8de7e0c4 of 2026-10-29, and that is the commit the replay built and ran (version 0.4.31; the script checks the replay's log against the fetched record). The checker is written in Rust and shares no code with the Lean kernel, which is what makes it a second opinion. Its author's book on external type checkers, linked from line 3 of the README, was not fetched.

9.6 Where we looked for an earlier statement of the title's observation

The title says the verified guarantee is weaker than trying all 2^n assignments at every practical size. We looked for an earlier statement of that observation in the texts this paper rests on, and in no others: the collection's `history.md`, its scope note `lean/docs/378.md`, the abstract of Sienicki and Sienicki (2026), the comparator README, the nanoda README, and the manuscript extract. The script searches each for the words such a statement would be likely to use ("practical", "brute", "trying all", "exhaustive", "2^n", "constants are", "concrete"). Outside the manuscript, none occurs. In the manuscript, "practical" occurs once (page 4, line 153), in a sentence about the practical value of SAT solvers on industrial formulas, a different sense; "exhaustive" occurs eleven times, as the motivation of SETH quoted above (line 170) and as the proof's own fallback for small inputs (lines 2480 and 2487, where the algorithm uses "exhaustive search" below the cutoff $n \geq 20$). The nearest earlier statements are the manuscript's own sentence that Lean verifies "the weaker exponent" (page 2, lines 63–67; Section 3) and the scope note: both say the verified exponent is the weaker one, and neither says at what sizes the verified bound exceeds 2^n, which is the title's point. Within these texts we found no prior statement of it. The search is bounded by that list: we did not search the literature at large, the working group's output, or the press.

9.7 What this does not claim

- **Nothing about the working group's findings.** The collection's log says the group "will examine and explain the proof"; we hold nothing it has written, and we neither anticipate nor contradict it.
- **The human audit does not cover this family.** It is cited for its method and its stated lesson about PDF extraction; its body was not fetched, and no sentence of it beyond the abstract is quoted.
- **The two hypothesis papers were not read.** They are cited as the manuscript cites them, and the hypotheses are stated as the manuscript states them (Section 6).
- **We did not run the comparator.** What we reproduced is stated in Section 4's Table 4.2; the comparator's statement match by exported declarations is not among it.
- **No claim that landrun, lean4export or nanoda is correct.** Each is quoted for what its README says it does; the replay's evidence is that two independent checkers agreed on this export, and the trust left in `lean4export` is stated in Section 5.
- **"No prior statement" is bounded by the list above.** Six texts, read with a script; not a literature search.

10 Conclusion

Placeholder. Item 09 drafts this section: what is claimed and what is not; every entry of `paper3/ETH_AMBIGUOUS.MD` that affects the text, listed as open by number. Ends with what this does not claim".

Data, code and proofs

Placeholder. Item 11 completes this statement: the repository, the scripts under `paper3/scripts/` that regenerate every number, the replay's commands and log hashes under `paper3/replay/`, the Lean names and the axiom audit under `paper3/lean/`, and the read-only clone of https://github.com/openai/math at commit `4eaee875`.

Acknowledgements

Placeholder. Item 11 writes the acknowledgements, with the disclosure of AI use in the form of `paper1/latex/sec8_ack.tex`.

References

ammkrn. nanoda_lib: an external type checker for Lean 4, written in Rust. https://github.com/ammkrn/nanoda_lib, 2026. Repository README, `master` branch, commit `8de7e0c4` (29 October 2026), version 0.4.31, fetched 5 November 2026.

Armin Biere, Tobias Faller, Katalin Fazekas, Mathias Fleury, Nils Froleyks, and Florian Pollitt. CaDiCaL 2.0. In *Computer Aided Verification (CAV 2024)*, volume 14681 of *Lecture Notes in Computer Science*, pages 133–152. Springer, 2024. Venue from memory; no DOI checked. The version run is 1.9.5, the one python-sat 1.9.dev15 bundles as `cadical195`.

Marco L. Carmosino, Jiawei Gao, Russell Impagliazzo, Ivan Mihajlin, Ramamohan Paturi, and Stefan Schneider. Nondeterministic extensions of the Strong Exponential Time Hypothesis and consequences for non-reducibility. In *7th Innovations in Theoretical Computer Science Conference (ITCS 2016)*, pages 261–270, 2016. Entry copied from the manuscript's reference list, extract lines 3686–3688; no DOI checked.

Supratik Chakraborty, Kuldeep S. Meel, and Moshe Y. Vardi. A scalable approximate model counter. In *Principles and Practice of Constraint Programming (CP 2013)*, volume 8124 of *Lecture Notes in Computer Science*, pages 200–216. Springer, 2013. Venue from memory; no DOI checked. Package pyapproxmc 4.4.0 (Mate Soos), https://github.com/meelgroup/approxmc.

Parinya Chalermsook, Marek Cygan, Guy Kortsarz, Bundit Laekhanukit, Pasin Manurangsi, Danupon Nanongkai, and Luca Trevisan. From Gap-Exponential Time Hypothesis to fixed parameter tractable inapproximability: Clique, Dominating Set, and more. *SIAM Journal on Computing*, 49(4):772–810, 2020. Entry copied from the manuscript's reference list, extract lines 3689–3691; no DOI checked.

Marek Cygan, Holger Dell, Daniel Lokshtanov, Dániel Marx, Jesper Nederlof, Yoshio Okamoto, Ramamohan Paturi, Saket Saurabh, and Magnus Wahlström. On problems as hard as CNF-SAT. *ACM Transactions on Algorithms*, 12(3):41:1–41:24, 2016. Entry copied from the manuscript's reference list, extract lines 3705–3707; no DOI checked.

Holger Dell, Thore Husfeldt, Dániel Marx, Nina Taslaman, and Martin Wahlén. Exponential time complexity of the permanent and the Tutte polynomial. *ACM Transactions on Algorithms*, 10(4):

21:1–21:32, 2014. Entry copied from the manuscript's reference list, extract lines 3708–3710; no DOI checked.

Irit Dinur. Mildly exponential reduction from Gap 3SAT to polynomial-gap Label-Cover. Technical Report TR16-128, Electronic Colloquium on Computational Complexity, 2016. Entry copied from the manuscript's reference list, extract lines 3711–3712; no DOI checked.

Alexey Ignatiev, Antonio Morgado, and Joao Marques-Silva. PySAT: A Python toolkit for prototyping with SAT oracles. In *Theory and Applications of Satisfiability Testing (SAT 2018)*, volume 10929 of *Lecture Notes in Computer Science*, pages 428–437. Springer, 2018. Venue from memory; no DOI checked. Package python-sat 1.9.dev15, https://github.com/pysathq/pysat.

Russell Impagliazzo and Ramamohan Paturi. On the complexity of k-SAT. *Journal of Computer and System Sciences*, 62(2):367–375, 2001. doi: 10.1006/jcss.2000.1727.

Russell Impagliazzo, Ramamohan Paturi, and Francis Zane. Which problems have strongly exponential complexity? *Journal of Computer and System Sciences*, 63(4):512–530, 2001. doi: 10.1006/jcss.2001.1774.

Lean FRO. Comparator: a trustworthy judge for Lean proofs. https://github.com/leanprover/comparator, 2026. Repository README, `master` branch, commit `865edc14` (14 October 2026), fetched 5 November 2026; archived, merged into Lean core as `lake comparator` at v4.35.0.

Pasin Manurangsi and Prasad Raghavendra. A birthday repetition theorem and complexity of approximating dense CSPs. In *44th International Colloquium on Automata, Languages, and Programming (ICALP 2017)*, volume 80 of *LIPIcs*, pages 78:1–78:15, 2017. Entry copied from the manuscript's reference list, extract lines 3759–3761; no DOI checked.

Stephan Mertens, Marc Mézard, and Riccardo Zecchina. Threshold values of random K-SAT from the cavity method. *Random Structures & Algorithms*, 28(3):340–373, 2006. Venue from memory; no DOI checked. Puts the 3-SAT threshold at 4.2667.

OpenAI. The Exponential Time Hypothesis is False. OpenAI Math Release preprint OAI:The-Exponential-Time-Hypothesis-is-False-October-25-2026, 2026.

Bart Selman, David G. Mitchell, and Hector J. Levesque. Generating hard satisfiability problems. *Artificial Intelligence*, 81(1–2):17–29, 1996. Venue from memory; no DOI checked.

Shubham Sharma, Subhajit Roy, Mate Soos, and Kuldeep S. Meel. GANAK: A scalable probabilistic exact model counter. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI 2019)*, pages 1169–1176, 2019. Venue from memory; no DOI checked. Package pyganak 2.8.0 (Mate Soos), https://github.com/meelgroup/ganak.

Mikołaj Sienicki and Krzysztof Sienicki. A human audit of OpenAI's AI-generated mathematical proofs, 2026. URL https://arxiv.org/abs/2608.14673. v3, 9 September 2026; 13 pages; the arXiv record spells the title "OpenAIs".

Mate Soos and Kuldeep S. Meel. BIRD: Engineering an efficient CNF-XOR SAT solver and its applications to approximate model counting. In *Proceedings of the 33rd AAAI Conference on*

Artificial Intelligence (AAAI 2019), pages 1592–1599, 2019. Venue from memory; no DOI checked. The ApproxMC version run descends from this one.

Richard Ryan Williams. Strong ETH breaks with Merlin and Arthur: short non-interactive proofs of batch evaluation. In *31st Conference on Computational Complexity (CCC 2016)*, volume 50 of *LIPIcs*, pages 2:1–2:17, 2016. Entry copied from the manuscript's reference list, extract lines 3810–3811; no DOI checked.

Appendix A Formal names

Every Lean name the paper cites, with its file under `lean/MOSPFormalization/SubexpPathwidth/` (the files are left where loop0014 wrote them, by the owner's decision recorded in `paper3/ETH_AMBIGUOUS.MD`, S3); the axiom audit is `paper3/lean/axiom_check.lean` with its committed log `paper3/lean/axiom_check.log`, and the byte-level rediff of the copy against the comparator file is `paper3/scripts/lean_rediff.py` with its log `paper3/lean/rediff.log`. Every row's declarations are on the three standard axioms `propext`, `Classical.choice`, `Quot.sound` or fewer; none depends on `sorryAx`. Nothing from the other study that shares the directory is cited.

	result	Lean names
The copy (`PrimitiveMachine.lean`, lines 31–221 = comparator lines 1–190 and 220, byte-identical; definitions only		
A.1	The comparator's vocabulary: formulas and their satisfying-assignment count, the canonical bit encoding and its length, the multitape machine with its step and start, the trace, the running-time scale $\rho(n)$ and the bound built from it, the count word, the located step and the visited space	`OAI.Algorithms.SubexponentialSAT.SAT.count`, `OAI.Algorithms.SubexponentialSAT.Encoding.encodeCNF`, `OAI.Algorithms.SubexponentialSAT.Encoding.formulaBitLength`, `OAI.Algorithms.SubexponentialSAT.Computation.PrimitiveMachine.step`, `OAI.Algorithms.SubexponentialSAT.Computation.Trace`, `OAI.Algorithms.SubexponentialSAT.SqrtLog.Statement.runtimeScale`, `OAI.Algorithms.SubexponentialSAT.SqrtLog.Statement.runtimeBound`, `OAI.Algorithms.SubexponentialSAT.SqrtLogSpace.countWord`, `OAI.Algorithms.SubexponentialSAT.SqrtLogSpace.locatedStep`, `OAI.Algorithms.SubexponentialSAT.SqrtLogSpace.visitedSpace`
The restatement (`Frame.lean`): two `def`s of a `Prop`, never proved, never axioms		

	result	Lean names
A.2	The comparator statement for width k as a named `Prop`, `SubexpCount` k: its body is the comparator theorem's body, identical after whitespace normalisation (1,073 characters, 23 lines)	`MOSPFormalization.SubexpPathwidth.SubexpCount`
A.3	The comparator theorem in full: for every $k \geq 2$, `SubexpCount` k	`MOSPFormalization.SubexpPathwidth.ExactCountAndSat`
What is proved about them (`Frame.lean`)		
A.4	A machine for width k is a machine for every width $k' \leq k$, with the same constants; and the unpacking of the "for every $k \geq 2$" form	`MOSPFormalization.SubexpPathwidth.SubexpCount.anti`, `MOSPFormalization.SubexpPathwidth.ExactCountAndSat.subexpCount`
A.5	The bound unfolds, by definition, to $C\,(L+1)^d\,2^{A\cdot\rho(n)}$	`MOSPFormalization.SubexpPathwidth.runtimeBound_eq`
A.6	$\rho(n) = o(n)$: the verified exponent is subexponential in the limit, from $(\ln y)^2/\sqrt{y} \to 0$	`MOSPFormalization.SubexpPathwidth.tendsto_log_sq_div_sqrt`, `MOSPFormalization.SubexpPathwidth.runtimeScale_isLittleO`
A.7	"f is a subexponential exponent" means $f = o(n)$; $A \cdot \rho(n)$ is one for every constant A	`MOSPFormalization.SubexpPathwidth.IsSubexpExponent`, `MOSPFormalization.SubexpPathwidth.isSubexpExponent_runtimeScale`
A.8	A run of the located (cell-monitoring) step projects to a run of the plain step with the same number of steps, so the comparator's time bound is a bound on the machine itself	`MOSPFormalization.SubexpPathwidth.LocatedCfg.monitor_config`, `MOSPFormalization.SubexpPathwidth.Trace.config_of_locatedStep`

Appendix B Technical detail behind the plain text

The body of the paper is written so that a reader with no background in complexity theory or in proof assistants can follow it. The passages that could not be made plain without losing precision are collected here, each under the heading of the section that cites it. Nothing in this appendix is new; it is the dense form of what the body says in words.

B.1 The definitions of the comparator file, line by line

This is the detail behind Section 2.2. Line numbers are those of `lean/ComparatorChallenges/SubexponentialSAT.lean` at commit `4eaee875`.

Formulas and the count (lines 13–51). A `Literal V` is a variable with a sign (lines 19–22); a `Clause V` is a `Finset` of literals and a `CNF V` a `Finset` of clauses (lines 24–25). An assignment is any function $V \to$ `Bool` (line 17). A literal evaluates to the Boolean "the assignment agrees with its sign" (lines 29–30), a clause to "some literal is true" (lines 32–33), a formula to "every clause is true" (lines 35–36). `WidthAtMost k F` says every clause has at most k literals as a set (line 38). Over a finite variable type, `satisfyingAssignments F` filters the universe of all assignments (lines 44–45), and `count F` is its cardinality (line 47): the number of satisfying assignments among all 2^n when $V =$ `Fin` n. The `Finset` choices change nothing in the meaning: a repeated clause or literal collapses, the empty clause is allowed and unsatisfiable, a tautological clause $\{x, \neg x\}$ is allowed and counts two toward the width. That is consistent with the manuscript's "Repeating a literal permits clauses with fewer than k literals" (p. 1, line 44).

The input encoding (lines 53–81). `encodeNat n` is n ones followed by a zero (line 55): unary. `frame` doubles a bit string, prefixing each bit with a one, and ends it with a zero (lines 57–58). `encodeList enc` prefixes each element's frame with a one and ends the list with a zero (lines 60–62). A literal becomes the number $2 \cdot \text{var} + [\text{positive}]$, at most $2n - 1$ (lines 64–65); a clause is its sorted list of literal numbers, a formula its sorted list of clauses (lines 67–71). `encodeCNF F` is `encodeNat n` followed by the formula's list of lists of unary numbers (lines 73–77), and `formulaBitLength F` is the length of that bit string (line 79). Two things follow. The number of variables n is part of the input, so a variable that occurs in no clause is still counted. And the length is polynomial: a clause of $j \le k$ literals costs $2j(4n+1)+4$ bits, so

$$L \;=\; \texttt{formulaBitLength}\, F \;\le\; n + 2 + m\,(2k(4n+1)+4) \;=\; O(k\,n\,m)$$

for m clauses. The solution itself proves $(L+1)^d \le (8k+7)^d\,(n+|F|+1)^{2d}$ (`SqrtLogSpace/Statement.lean`, lines 9–14). Emulating the definitions (`reports/scratch/eth_audit/check_encoding.py`), a formula with $n = 125$, $m = 500$, $k = 4$ encodes to $L = 1{,}006{,}223$ bits and one with $n = 1{,}000$, $m = 4{,}000$, $k = 3$ to 47.9 million, both under the bound. Nothing in the encoding is exponential; the unary numbers are linear in n.

The machine (lines 83–128). `PrimitiveMachine` (lines 92–99) is a finite object: a number `tapeCount` of tapes and a number `stateCount` of states, an input tape and a start state, a table `scan` saying which tape each state reads, and a table `transition` from `Fin stateCount` × `Option Bool`—a domain of $3 \cdot$ `stateCount` points—into either a halting answer (a `Bool`) or a next state with one action. The action type is Mathlib's `Turing.TM0.Stmt (Option Bool)`: move the head left or right, or write one of the three symbols `none`, `some false`, `some true`, five values in all (`Mathlib/Computability/TuringMachine/PostTuringMachine.lean` lines 118–120 and `Tape.lean` lines 421–423 at Mathlib `d13f23b7`, the clone's pinned revision). A configuration `Cfg` (lines 103–105) is a state, halted or not, and one Mathlib `Tape` per tape index (a head symbol and two blank-padded halves, `Tape.lean` lines 408–414, blank = `none`). `step` (lines 107–119) returns `none` in a halted state; otherwise it reads the head of the one tape the

state scans, looks the pair up in the table, and either halts with the answer, or writes one symbol or moves that one head by one cell and changes state. `init` (lines 121–124) puts the input on the input tape with the head at its first bit and every other tape blank (`Tape.mk₁`, `Tape.lean` line 494). A Lean function on a finite domain is, whatever term defines it, a finite table; so a step is one lookup in a table of $3 \cdot$ `stateCount` rows, and no field of the structure can carry unbounded computation.

Traces (lines 87–90). `Trace step t a c` is an inductive proposition with two constructors: `refl`, a trace of length 0 from a to a itself, and `cons`, which extends a trace from b by one successful step $a \mapsto b$. So it holds exactly when c is reached from a by t successful applications of `step`, and a trace of length 0 to a different configuration is impossible. Since `step` is a function, the run is deterministic.

Space monitoring (lines 147–189). `LocatedCfg` (lines 149–153) pairs a configuration with, per tape, the head's current position and the least and greatest positions seen so far, as integers; `locatedInit` starts every head at 0 with the input tape's range covering the input (lines 155–159); `headDisplacement` is -1, $+1$ or 0 for a move left, a move right or a write (lines 161–164); `monitor` (lines 166–182) applies the same table lookup to update the position and the running minimum and maximum of the scanned tape, and nothing else; `locatedStep` is `step` on the configuration followed by `monitor` (lines 184–186). `visitedSpace` (lines 188–189) is the sum over tapes of upper $-$ lower $+ 1$: the number of cells whose head position was ever visited. `countWord a` is the binary expansion of a, least significant bit first, with `[false]` for 0 (line 147), and `toNat` (lines 7–9) decodes such a word.

The exponent and the bound (lines 130–141).

```
runtimeScale n = n * (Real.log (Real.log (n + 16)))^2 /
Real.sqrt (Real.log (n + 16))
```

(lines 132–134, transcribed; the casts to $\mathbb{R}$ omitted), that is

$$\rho(n) \;=\; \frac{n\,(\ln\ln(n+16))^2}{\sqrt{\ln(n+16)}},$$

with `Real.log` the natural logarithm. The manuscript fixes the same base: "Throughout, log denotes the natural logarithm" (p. 1, lines 48–49; again at p. 7, lines 326–327). The shift by 16 keeps $\ln\ln$ defined and positive at every $n \geq 0$ ($\ln 16 = 2.77$), so the theorem can be stated for all n with no proviso. Then

```
runtimeBound C A d L n = C * (L + 1)^d * 2^(A * runtimeScale n)
```

(lines 136–139, transcribed), a real number, with the real-exponent power.

B.2 The theorem, with its quantifiers

This is the detail behind Section 2.3. Lines 194–217 of the comparator file state, for every k with $2 \le k$:

$\exists$ `machine`, `countTape`, `answerTape`, $C, A, S \in \mathbb{R}$, $d, e \in \mathbb{N}$,
$0 < C \ \wedge\ 0 < A \ \wedge\ 0 < S \ \wedge\ 0 < d \ \wedge\ 0 < e \ \wedge$
$\forall\, n \in \mathbb{N},\ \forall\, F : \texttt{CNF}\ (\texttt{Fin}\ n),\ \texttt{WidthAtMost}\ k\ F\ \rightarrow$
$\exists\, \texttt{time} \in \mathbb{N},\ \texttt{final} : \texttt{LocatedCfg machine},$
`Trace (locatedStep machine) time (locatedInit machine (encodeCNF F)) final`
(lines 201–202)
$\wedge$ the count tape holds `countWord (count F)` (203–204)
$\wedge$ the answer tape holds the bit $[0 <$ `count` $F]$ (205–206)
$\wedge$ the halting state carries that bit (207)
$\wedge$ `toNat (countWord (count F)) = count F` (208)
$\wedge\ |\texttt{countWord}\ (\texttt{count}\ F)| \le n + 1$ (209)
$\wedge$ `time` $\le C\,(L+1)^d\, 2^{A\,\rho(n)},\ L =$ `formulaBitLength F` (210–211)
$\wedge\ \forall$ prefix of the trace, `visitedSpace` $\le S\,(L+1)^e$ (212–217).

The machine and the five constants are quantified after k and before n and F; inside the universal quantifier only `time` and `final` are chosen. This is the standard $O_k(1)$ reading: for each k one fixed finite machine and one set of constants serve every formula of every size. The table's size is $O_k(1)$ in the same sense, with no explicit function of k. There is no lower bound on n: the manuscript's Theorem 1.1 and Proposition 6.4 are stated for $n \ge 20$ (p. 2, lines 56–57; p. 43, lines 2474–2475), and the solution's auxiliary `exact_count_eth` keeps $20 \le n$ (`SqrtLogSpace/Final.lean`, lines 104–109); the exported theorem does not, and the $+16$ in $\rho(n)$ is what makes that possible.

The proof's own constants, read for their values only: at the point where the large-input run is assembled, the exponent constant is set to $A = 256 + A_t$ with $A_t \ge 1$ taken from a leaf contract (`SqrtLogSpace/NativeLargeEntry.lean`, line 51; `SqrtLogSpace/NativeLeafContract.lean`, lines 792–793), and $1 \le C$, $1 \le A$ are in the large-input run's own statement (`SqrtLogSpace/NativeLargeEntry.lean`, line 12). The witness machine is compiled from a program over 80 one-bit registers and 1,220 tapes, with `countTape` $= 256$ and `answerTape` $= 1216$ (`SqrtLogSpace/Final.lean`, lines 98–101); its state count is a finite type's cardinality that nothing computes. The solution's internal ρ (`SqrtLogSpace/Asymptotics.lean`, lines 10–11) is in base 2 and is related to `runtimeScale` by `runtimeScale` $n \le 4\,\rho_2(n)$ for $n \ge 20$ (lines 190–191); the base changes the exponent by a constant factor that A absorbs.

The clause-count bound used in Sections 2 and 6: a `Finset` of clauses of width at most k over n variables has at most $\sum_{j \le k} \binom{2n}{j} = O(n^k)$ members (2,604,376 at $n = 125$, $k = 3$), so $m = O(n^k)$, $L = O(k\,n\,m) = O(n^{k+1})$, and the whole bound is $n^{O_k(1)}\, 2^{A\rho(n)}$, which is $2^{o(n)}$ in n alone because $\rho(n)/n \to 0$ (Appendix A, A.6).

B.3 The spine of the proof, with every line

This is the detail behind Section 4.5. Unqualified paths are relative to `lean/OAI/Algorithms/SubexponentialSAT/` in the clone.

The machine. The theorem's machine is `GuardedCompiler.primitive program` with the count tape `256` and the answer tape `1216` (both `: Fin 1220`; `SqrtLogSpace/Final.lean`, line 100), where `program` is `Native.countingProgram cutoff k (NativeLeafProgram.command k)` (line 40), a `BitProgram.Program 80 1220 labels` (line 11): a first-order program over 80 one-bit registers, 1,220 Boolean stacks and a finite label set, with twelve instruction kinds (`BitProgram.lean`, lines 10–26). `primitive` (`SqrtLogSpace/GuardedCompiler.lean`, line 131) is a `noncomputable abbrev`: it takes the transition table `transition p` (lines 30–47, one case per instruction) and the stack selector `selected p` (lines 14–17), feeds them to the stack compiler's controller, and renames the finite state type to `Fin stateCount` through `Fintype.equivFin` (line 132). The machine is thus a concrete table defined by pattern matching on the program, composed with a classical renaming of states; the theorem is existential, so the noncomputability costs it nothing.

The time bound, and the constants. For `cutoff < n`, `NativeLargeEntry.run` (`SqrtLogSpace/NativeLargeEntry.lean`, lines 11–33) supplies a trace of length at most `runtimeBound C A d L n` (line 27) with $1 \le C \wedge 1 \le A$ (line 12). Its exponent constant is `256 + At` (line 51), where `At` comes from the leaf program's uniform contract (`NativeLeafContract.uniform_contract`, line 39; the contract gives $1 \le$ `At`, `SqrtLogSpace/NativeLeafContract.lean`, line 793), and its cutoff is `max 20 (max Nleaf Ns)` (line 50), with `Ns` obtained non-constructively from an eventual inequality (`Filter.eventually_atTop`, line 48). The $2^{A \cdot \texttt{runtimeScale}\, n}$ factor is assembled by `runtime_of_tree_and_leaf` (line 46) from the DFS visit bound of `Native.outer_from_setup` (`SqrtLogSpace/NativeOuter.lean`, lines 53–56: visits at most a square polynomial in n times $2^{n/\texttt{sparsifySchedule}\, n}$) and the leaf time. For $n \le$ `cutoff`, `Native.entry_small_primitive` (`SqrtLogSpace/NativeEntry.lean`, lines 81–100) gives `time` $\le 28700\,(L+1)^2$ (line 92) through the finite scanner of `SqrtLogSpace/FiniteFallback.lean`. The two branches are folded by `runtime_with_fallback` and `space_with_fallback` (`SqrtLogSpace/NativeOuterBudgets.lean`, lines 297 and 317) into one triple, and the witnesses the final theorem exhibits are `C + 28700`, `A + 1`, `S + 5022741`, `d + 2`, `e + 2` (`SqrtLogSpace/Final.lean`, line 41). So the proof as written has an exponent constant of at least 258 in front of $\rho(n)$, a polynomial degree `d` that depends on k (line 51: `db` $+ (4(k-2)+2) +$ `df` $+ 2$) and a leading constant of at least 28,701; none of these is in the statement, which asserts only that they are positive, and none of `At`, `db`, `df` is computed anywhere.

Where a hidden cost model would live, and why it cannot. The compile lemmas conclude in the comparator's own vocabulary: `GuardedCompiler.primitive_execution` (`SqrtLogSpace/GuardedCompiler.lean`, lines 158–162) turns a source run of `n` steps into `RunsIn (primitive p).step (7 * n)`, with the overhead `7` proved as a theorem (`compiler_overhead`, lines 152–153), and `CompileSpace.compile_word_answer_space` (`SqrtLogSpace/CompileSpace.lean`, lines 125–143) produces a `Computation.Trace (locatedStep (primitive p)) time ...` with `time` $\le 7 \cdot$ `sourceSteps` and `visitedSpace intermediate` $\le s($`bound` $+$ `input.length` $+ 15)$ on every prefix (lines 134, 135 and 143); `primitive_located_step` (`SqrtLogSpace/GuardedCompiler.lean`, line 600) is the per-step form. `Trace`, `locatedStep` and `visitedSpace` are the challenge's defini-

tions (its lines 87–90 and 149–189), and `time` in the final theorem is literally the length of a `Computation.Trace` of `PrimitiveMachine.step` (its line 201). No cost function of the compiler's appears in the statement; a wrong cost model would make these lemmas unprovable, not the theorem false.

Correctness of the count. `Native.outer_from_setup` (`SqrtLogSpace/NativeOuter.lean`, lines 50–51) yields `final.stacks ABI.countStack = countWord (SAT.count F)` and `final.stacks ABI.answerStack = [decide (0 < SAT.count F)]` from the canonical DFS execution, and `Native.counting_primitive_from_entry` (line 73) transports both to the primitive machine's tapes; `countWord_toNat` and `countWord_count_length` supply the decoding and length clauses of the statement.

The statement match, counted. The solution file does not import the challenge file: `SqrtLogSpace/Final.lean` begins (lines 1–3) with `NativeLargeEntry`, `SqrtLogSpace.Statement` and `SqrtLog.StatementCanonical`. The library redeclares every definition the statement uses under the same fully qualified names, in seven files: the SAT vocabulary in `SAT.lean`, the bit arithmetic in `BinaryArithmetic.lean`, the nine encoding functions in `Encoding.lean`, the machine in `Computation.lean`, `runtimeScale` and `runtimeBound` in `SqrtLog/StatementScale.lean`, the space instrumentation in `SqrtLogSpace/VisitedTape.lean` and `countWord` in `SqrtLogSpace/EncodingBounds.lean` (line 17, the challenge's line 147). The definition diff compares 35 declarations: 33 are identical character for character after whitespace normalisation, and the remaining two, `runtimeScale` and `runtimeBound`, differ only in that the challenge writes `noncomputable def` where the library opens a `noncomputable section` (`SqrtLog/StatementScale.lean`, line 15). The theorem statement itself, from `theorem exact_count_and_sat` to `:= by`, is identical in the two files, 1,353 characters raw and 1,124 after normalisation, at line 194 of the challenge and line 74 of `SqrtLogSpace/Final.lean`; the challenge's proof is the single word `sorry` at its line 218.

B.4 The keyword sweep, item by item

This is the detail behind Section 4.3. The only `set_option`s in the closure are two `set_option quotPrecheck false` (`SqrtLog/LeafProgramPreparedRecoveryQuery.lean`, line 15, and `SqrtLog/LeafProgramPreparedRecoveryWitness.lean`, line 14), which switch off a pre-check on local notation and have no bearing on soundness; there is no `maxHeartbeats`, `maxRecDepth` or `autoImplicit` option anywhere in the closure, and the lakefile sets `autoImplicit` to `false` for the whole package (`lean/lakefile.lean`, line 9). Nine declarations are `opaque`, and every one carries a body: three recovery functions (`queryRecovery` at line 59 of `SqrtLog/LeafProgramPreparedRecoveryQuery.lean`, with its `:= by` at line 86; `rawRecovery` and `fixedRecovery` at lines 178 and 198 of `SqrtLog/LeafProgramPreparedRecoveryWitness.lean`) and six `private opaque` constants of a subtype `{ports // ports = ...}`, each with an explicit value (line 22 of `SqrtLog/LeafProgramQueryCleanup.lean` and five others). An `opaque` with a body is a definition whose unfolding is hidden from elaboration; the kernel still checks the body against the type. The 1,048 `noncomputable` declarations, the eight `open Classical`, and the four

explicit uses each of `Classical.choice` and `propext` are the classical reasoning the comparator permits. The 148 `instance`s and 40 `local instance`s are `Fintype`, `Decidable` and algebraic instances; an instance cannot be "inconsistent" without an axiom, and there is none. The 14,034 lines with `decide` are kernel-evaluated decision procedures, not the native evaluator. Nine `termination_by` and five `decreasing_by` clauses are the recursions the kernel checks.

The lakefile's two hooks: a `run_cmd` block (lines 290–304) that evaluates dependency declarations with `unsafe env.evalConstCheck` and calls `preparePatchedDependencies`, which clones and patches eleven dependencies, and a `post_update` hook (line 306 onward) that verifies or applies the patches for the rest. They touch only `.lake/packages`, and only non-Mathlib packages. The 23 compatibility patches under `lean/patches/` are for third-party packages (`AINTLIB`, `PrimeNumberTheoremAnd`, `carleson`, `iut`, ...) that this closure does not import. The registry `lean/formalization.yaml` (lines 1683–1685) points at the same declaration and file as the comparator's JSON.

B.5 The hypotheses in quantifier form

This is the detail behind Section 6.2. The property used throughout: for each fixed $k \geq 2$ there is one deterministic machine which, on the canonical encoding of any `WidthAtMost` k formula F on n variables (comparator line 38), halts with `countWord (count F)` on its count tape (lines 203–204, 147), the bit $[0 < \texttt{count}\ F]$ on its answer tape and in its halting state (lines 205–207), within $C\,(L+1)^d\,2^{A\rho(n)}$ steps (lines 210–211). Since $\rho(n)/n \to 0$ (Appendix A, A.6), for every $\delta > 0$ there is an $n_0(k,\delta)$ beyond which the bound is below $2^{\delta n}$; below n_0 the bound is a constant depending on k and δ. So the running time is $O_{k,\delta}(2^{\delta n})$ for every $\delta > 0$, and $2^{\delta n}(n+m)^{O_k(1)}$ with the polynomial written out.

ETH. In quantifiers: there is a $\delta > 0$ such that no deterministic algorithm decides 3-SAT within $2^{\delta n}\,\mathrm{poly}(n+m)$ steps on all inputs. Its negation: for every $\delta > 0$ there is one. The verified statement at $k = 3$ supplies, for every $\delta > 0$, the same machine, whose answer bit is the decision, within $O_\delta(2^{\delta n})\,\mathrm{poly}(n+m)$ steps. This holds on every input of every size, so it refutes the hypothesis under either reading of "$2^{\Omega(n)}$" (a lower bound at infinitely many n, or at all large n). In the vocabulary of Impagliazzo and Paturi (2001), where s_k is the infimum of the δ for which k-SAT has an $O(2^{\delta n})$ algorithm and ETH is $s_3 > 0$: the verified statement gives $s_k = 0$ for every $k \geq 2$, by one algorithm per k.

SETH. The manuscript's form: for every $\varepsilon \in (0,1)$, there is a fixed k for which no deterministic algorithm solves k-SAT in $2^{(1-\varepsilon)n}(n+m)^{O(1)}$ time. The negation: there is an $\varepsilon \in (0,1)$ such that for every fixed k there is a deterministic $2^{(1-\varepsilon)n}(n+m)^{O(1)}$ algorithm. The verified statement gives more: for every $\varepsilon \in (0,1)$ and every $k \geq 2$, the machine for k runs within $C_k(L+1)^{d_k}\,2^{A_k\rho(n)}$ steps, and $A_k\rho(n) \leq (1-\varepsilon)n$ for all $n \geq n_0(k,\varepsilon)$ because $\rho(n) = o(n)$; below n_0 the exponential factor is a constant. So each k has a $2^{(1-\varepsilon)n}(n+m)^{O_k(1)}$ algorithm, and the polynomial's degree depending on k is exactly what the hypothesis's "for a fixed k" permits. For the record, SETH implies ETH: if 3-SAT had a $2^{o(n)}$ algorithm then, by the Sparsification Lemma of Impagliazzo et al. (2001), which reduces k-SAT on n variables to $2^{\varepsilon n}$ instances with $O_{k,\varepsilon}(n)$ clauses and so, after the clause-splitting reduction, to 3-SAT on $O_{k,\varepsilon}(n)$ variables, so would k-SAT for every fixed k.

Gap-ETH. The negation of the manuscript's form: for every $\delta, \gamma > 0$ there is an $O(2^{\gamma n})$ algorithm that separates a satisfiable 3-CNF from one in which every assignment violates at least a δ fraction of the clauses, already at $m = O(n)$. Exact decision separates them, since a formula in which every assignment violates a positive fraction of the clauses is unsatisfiable; and the verified machine at $k = 3$ decides within $n^{O(1)}\, 2^{A\rho(n)} = O_\gamma(2^{\gamma n})$ steps for every $\gamma > 0$, at every m (at $m = O(n)$ the polynomial is $O(n^{2d})$). The promise is never used: the machine answers correctly on every input, promised or not, with probability 1, which refutes the randomised form as well.

NSETH. The negation of the manuscript's form: there is an $\varepsilon > 0$ such that every fixed k has a nondeterministic $2^{(1-\varepsilon)n}(n+m)^{O(1)}$ algorithm for unsatisfiability. The verified machine's answer bit is `false` exactly when `count` $F = 0$ (lines 205–207), so it decides unsatisfiability deterministically within the SETH bound above for every ε and every k; and a deterministic machine is a nondeterministic one whose only computation path is the deterministic run, so every acceptance convention for nondeterministic time is met.

#ETH. The negation: for every $\delta > 0$ there is a deterministic $O(2^{\delta n})$ algorithm that outputs the exact count. The verified machine at $k = 3$ halts with `countWord (count F)` on its count tape, a binary word of length at most $n + 1$ that decodes to `count` F (lines 203–204, 208–209), within the same bound.

Shape (S) in detail. For a theorem "SETH $\Rightarrow$ no $(c-\varepsilon)^k$ algorithm for Y", a $(c-\varepsilon)^k$ algorithm for Y obtained from the verified statement would need the exponent of the SAT instance to be below $k \log_2 c$ by a constant factor, and the verified statement's exponent is $A\rho(n')$ in the variable count n' of the SAT instance with A unknown; no value of A is on record below 257, and no base c is derived from the verified statement for any Y.

B.6 How the two guarantee curves are scaled

This is the detail behind Section 7.4. Both guarantees are taken at constant 1 (the polynomial factors, C, A and d set aside) and both are multiplied by the same factor $t_{20}/2^{20}$, where $t_{20} = 1.28 \times 10^{-4}$ s is CaDiCaL's median at $n = 20$. The factor is one number for both curves, so their ratio is the ratio of the guarantees themselves, $2^{\rho(n)-n}$: below 1 for $n < 50$, equal to 1 at $n = 50$ up to the rounding of $\rho(50) = 50.13$, and above 1 from there (2.8×10^3 at $n = 100$ and 4.0×10^{51} at $n = 500$). A reader who prefers each curve scaled to the $n = 20$ point on its own would multiply $2^{\rho(n)}$ by a further $2^{20-\rho(20)} = 2^{2.79}$ and see the two curves touch at $n = 20$ and the verified one above the trivial one from there on; the common factor keeps the crossing where the statement puts it (`ETH_AMBIGUOUS.MD`, 06a). In that scale the brute-force guarantee reaches one second near $n = 33$ and the verified guarantee near $n = 35$; at $n = 60$ the measured CaDiCaL median is 3×10^{-4} s against 2^n at 1.4×10^8 s and $2^{\rho(n)}$ at 5.4×10^8 s, 11.6 and 12.2 decades respectively; at $n = 500$ the decision median is above the budget (8 of 20 finished within 60 s), against $10^{140.6}$ s and $10^{192.2}$ s. Regenerate: `python3 -I paper3/scripts/practitioner.py`.